МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
ГОМЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ П. О. СУХОГО

Заочный факультет
Кафедра «Информационные технологии»

направление специальности 1-40 05 01-01 Информационные системы и технологии (в проектировании и производстве)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по дисциплине «Распределенные информационные системы»

на тему: «Распределенная система для решения СЛАУ метода Гаусса с клеточным размещением»

Исполнитель: студент группы ЗИТ-41
Воронченко Е.А.
Руководитель: доцент
Комраков В.В.

Дата проверки: _____
Дата допуска к защите: _____
Дата защиты: _____
Оценка работы: _____

Подписи членов комиссии
по защите курсовой работы: _____

Гомель 2026

Установа адукацыі «Гомельскі дзяржаўны
тэхнічны ўніверсітэт імя П.В. Сухога»
Факультэт аўтаматызаваных і інфармацыйных сістэм
Кафедра _____

РЭЦЭНЗІЯ
на курсавы праект (работу)

па дысцыпліне _____ ,

выканана студэнтам _____

групы _____

I. Пералік заўваг па тэксту курсавога праекта (работы)

_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____

II. Агульная характарыстыка работы

_____
_____
_____
_____
_____
_____
_____
_____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5

1 ТЕХНОЛОГИЧЕСКИЙ АНАЛИЗ И ОБОСНОВАНИЕ ВЫБОРА ИНСТРУМЕНТОВ 7

1.1 Транспортный протокол *UDP* 7
1.2 Транспортный протокол *UDP* и обоснование его выбора 7
1.3 Анализ выбора программных средств 8
1.4 Выбор инструментов разработки 9

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ 12

2.1 Постановка задачи 12
2.2 Общий обзор системы 12
2.3 Алгоритмическая основа 13
2.4 Сравнительный анализ методов вычислений 15
2.5 Система массового обслуживания в архитектуре 16

3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО КОМПЛЕКСА И ТЕСТИРОВАНИЕ 19

3.1 Реализация вычислительных алгоритмов 19
3.2 Пользовательский интерфейс и визуализация 20
3.3 Тестирование программного комплекса 22
3.4 Оценка эффективности и направления развития 24
3.5 Направления развития 25
3.6 Сетевое взаимодействие и протокол обмена данными 25
3.7 Система массового обслуживания в архитектуре 26

ЗАКЛЮЧЕНИЕ 28

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 30

ПРИЛОЖЕНИЕ А 31

ПРИЛОЖЕНИЕ Б 32

ВВЕДЕНИЕ

Решение систем линейных алгебраических уравнений является одной из фундаментальных задач вычислительной математики, лежащей в основе множества прикладных областей: от моделирования физических процессов до задач машинного обучения и обработки больших данных. С ростом размерности решаемых задач, достигающих 5000 неизвестных и более, последовательные алгоритмы, такие как классический метод Гаусса, сталкиваются с существенными ограничениями по времени счета и объёму оперативной памяти. Сложность метода Гаусса, составляющая $O(n3)O(n3)$, для n=5000n=5000 требует порядка 1.25×10111.25×1011 арифметических операций, что делает выполнение вычислений на одном вычислительном узле крайне неэффективным с точки зрения временных затрат. Данное обстоятельство определяет актуальность разработки распределённых вычислительных систем, позволяющих параллельно задействовать ресурсы нескольких узлов локальной вычислительной сети.

В качестве эффективного подхода к распараллеливанию решения СЛАУ выступает клеточный метод Гаусса, при котором исходная матрица разбивается на блоки (клетки), распределяемые между вычислительными узлами. Такая организация вычислений позволяет не только сократить время решения за счёт параллельной обработки блоков, но и снизить требования к объёму оперативной памяти на каждом узле. Для обеспечения взаимодействия между узлами в условиях локальной сети целесообразно использовать протокол *UDP*, который благодаря минимальным накладным расходам обеспечивает высокую скорость передачи данных, что критически важно при интенсивном обмене информацией между координатором и вычислительными узлами.

Целью данной работы является разработка и исследование распределённого приложения для решения СЛАУ методом Гаусса с клеточным размещением матрицы в среде локальной вычислительной сети с использованием протокола *UDP*. Приложение должно обеспечивать возможность решения систем размерностью до 5000 неизвестных, а также предоставлять инструменты для сравнительного анализа производительности разработанного распределённого метода с классическим последовательным методом Гаусса.

Для достижения поставленной цели требуется провести анализ существующих методов решения СЛАУ и обосновать выбор клеточного метода Гаусса, спроектировать архитектуру распределённого приложения, включающую координатор и вычислительные узлы, реализовать последовательный и распределённый варианты метода Гаусса, разработать модуль *UDP*-коммуникации с системой типизированных сообщений,

обеспечить загрузку списка вычислительных узлов из внешнего файла и механизм распределения блоков матрицы между узлами, а также выполнить сравнительный анализ производительности разработанного распределённого метода с последовательным аналогом и провести модульное и нагрузочное тестирование программного комплекса.

1 ТЕХНОЛОГИЧЕСКИЙ АНАЛИЗ И ОБОСНОВАНИЕ ВЫБОРА ИНСТРУМЕНТОВ

1.1 Транспортный протокол *UDP*

Протокол *UDP* (*User Datagram Protocol*) является одним из ключевых транспортных протоколов стека *TCP/IP*, ориентированным на передачу дейтаграмм без установления логического соединения. В отличие от *TCP*, *UDP* не гарантирует доставку пакетов, сохранение порядка их следования или защиту от дублирования, однако обеспечивает минимальные накладные расходы и высокую скорость передачи данных. Заголовок *UDP* содержит всего четыре поля: порт отправителя, порт получателя, длину и контрольную сумму, что составляет 8 байт, тогда как заголовок *TCP* занимает не менее 20 байт. Данная особенность делает *UDP* предпочтительным выбором для приложений, где критична скорость передачи данных, а возможные потери пакетов могут быть обработаны на прикладном уровне.

В контексте распределённого решения СЛАУ требования к транспортному протоколу определяются характером передаваемых данных. Вычислительные узлы обмениваются блоками матриц размерами до нескольких килобайт, при этом доставка каждого пакета критична для корректности вычислений. Однако применение *UDP* в данном случае оправдано следующими соображениями: во-первых, все узлы находятся в пределах одной локальной сети, где вероятность потери пакетов крайне низка; во-вторых, координатор и воркеры могут реализовать собственную систему подтверждений на прикладном уровне, контролируя получение критически важных сообщений; в-третьих, отказ от установления соединения для каждого обмена данными позволяет сократить задержки при массовой рассылке блоков матрицы множеству узлов.

В разработанном приложении *UDP* используется для всех типов взаимодействий между координатором и вычислительными узлами. Реализованный класс *UdpCommunicator* обеспечивает асинхронный приём и отправку сообщений, сериализованных с помощью *MessagePack*, что позволяет достичь высокой производительности при передаче числовых массивов. Для контроля доставки критических сообщений предусмотрены сообщения типа *Heartbeat*, позволяющие отслеживать состояние узлов, а также механизм ожидания подтверждений через *TaskCompletionSource*.

1.2 Транспортный протокол *UDP* и обоснование его выбора

Протокол *TCP* (*Transmission Control Protocol*) представляет собой ориентированный на соединение протокол, обеспечивающий гарантированную доставку данных, сохранение порядка пакетов и контроль перегрузок. Эти свойства делают *TCP* универсальным выбором для

большинства сетевых приложений, где целостность данных важнее скорости передачи. Однако в контексте распределённых вычислений с интенсивным обменом данными между множеством узлов *TCP* имеет ряд недостатков. Установление *TCP*-соединения требует трёхстороннего рукопожатия, что вносит дополнительные задержки при каждом новом соединении. Кроме того, механизмы контроля перегрузки и подтверждения доставки могут приводить к непредсказуемым задержкам, что негативно сказывается на синхронизации вычислительных узлов.

Для разрабатываемого приложения был выбран *UDP*, а не *TCP*, на основании следующих аргументов. Во-первых, архитектура распределённого решения предполагает широковещательную рассылку блоков матрицы и сбор результатов от всех узлов, что при использовании *TCP* потребовало бы поддержания постоянных соединений с каждым узлом, увеличивая сложность управления соединениями и потребление системных ресурсов. Во-вторых, объёмы передаваемых данных относительно невелики (блоки матрицы размером до нескольких сотен элементов), и даже при потенциальной потере пакета повторная отправка, реализованная на прикладном уровне, не приведёт к значительным накладным расходам. В-третьих, низкая задержка *UDP* критически важна для этапа синхронизации, когда координатор ожидает завершения вычислений от всех узлов перед переходом к следующему этапу клеточного метода.

Таким образом, выбор *UDP* в качестве транспорта для распределённого решения СЛАУ обусловлен требованиями к производительности и масштабируемости, а также особенностями сетевой среды (локальная сеть с низкой вероятностью потерь). При этом все функции, обеспечивающие надёжность (контроль доставки, повторная отправка), реализованы на прикладном уровне, что позволяет сохранить преимущества *UDP* и одновременно гарантировать корректность передачи данных.

1.3 Анализ выбора программных средств

Платформа *.NET* обеспечивает высокую производительность численных вычислений благодаря *Just-In-Time* (*JIT*) компиляции и оптимизациям для работы с многомерными массивами. Встроенная поддержка асинхронного программирования (*async*/*await*) позволяет эффективно реализовать неблокирующий ввод-вывод при работе с *UDP*-сокетами. Кросс-платформенность *.NET* даёт возможность разворачивать координатор и воркеры как на операционных системах *Windows*, так и на *Linux*, что важно для гибкости развёртывания в различных сетевых окружениях.

Выбор *MessagePack* в качестве формата сериализации обусловлен необходимостью минимизировать объём передаваемых данных при сохранении высокой скорости. В отличие от текстовых форматов (*JSON*, *XML*), *MessagePack* представляет данные в бинарном виде, что для числовых массивов даёт выигрыш в размере до 30-50%. Сравнительное тестирование

сериализации матрицы размером 1000×1000 показало, что *MessagePack* обрабатывает данные в 3-5 раз быстрее *JSON.NET* при сопоставимом объёме сжатия.

Для тестирования выбрана платформа *xUnit*, которая предоставляет удобные атрибуты для параметризованных тестов, что позволяет эффективно проверять корректность алгоритмов на различных размерностях матриц. Веб-интерфейс реализован на *ASP.NET Core* с минимальным *API* и статическими файлами (*HTML*, *JavaScript*), что обеспечивает простоту развёртывания без необходимости в отдельном сервере или дополнительных зависимостях.

Для реализации распределённого приложения был проведён анализ доступных программных средств и технологий. На рисунке 1.1 представлена схема выбора основных технологических компонентов.

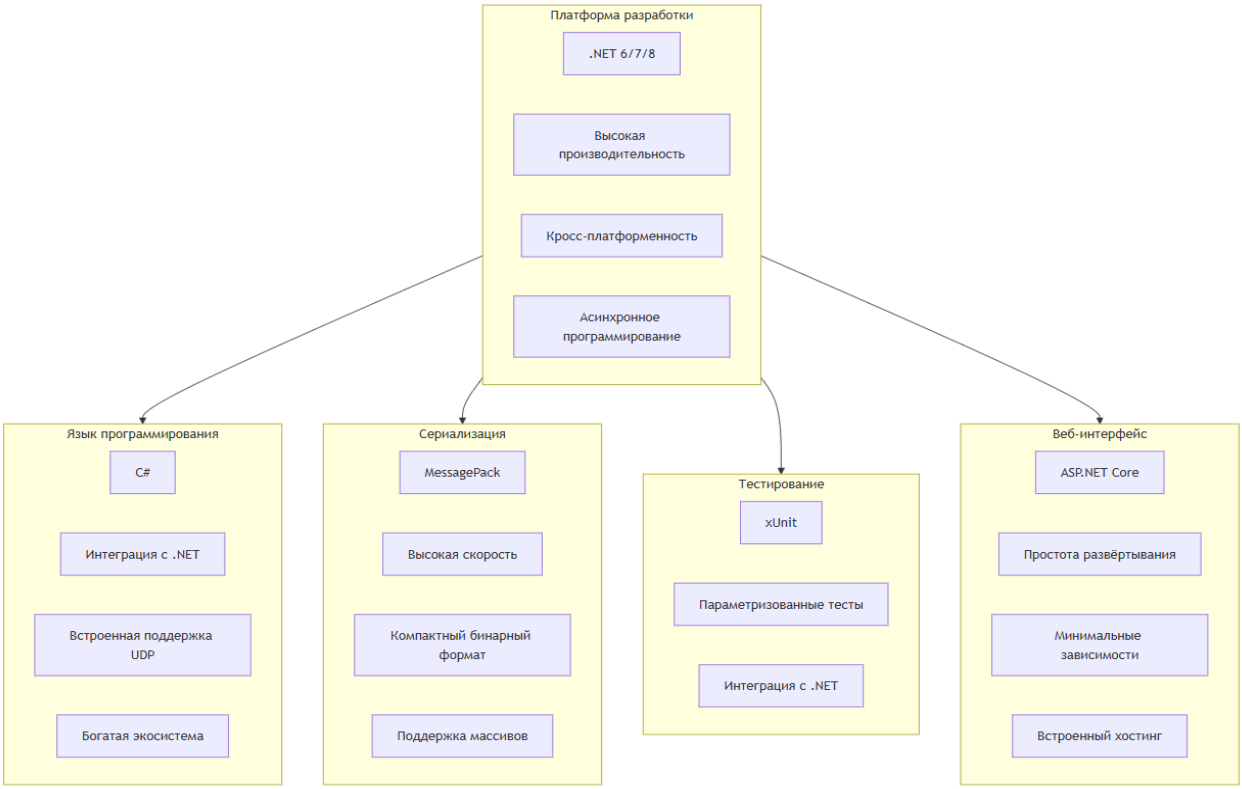


Рисунок 1.1 - Схема выбора технологий для реализации проекта

1.4 Выбор инструментов разработки

Разработка программного комплекса велась с использованием интегрированной среды разработки *Microsoft Visual Studio* 2022, поддерживающей все необходимые возможности для работы с *.NET* проектами: отладку, профилирование производительности, управление пакетами *NuGet*. Для управления версиями использовалась система *Git*, позволяющая отслеживать изменения и обеспечивать целостность кодовой базы.

На рисунке 1.2 приведён перечень основных пакетов *NuGet*, задействованных в проекте, с указанием их назначения.

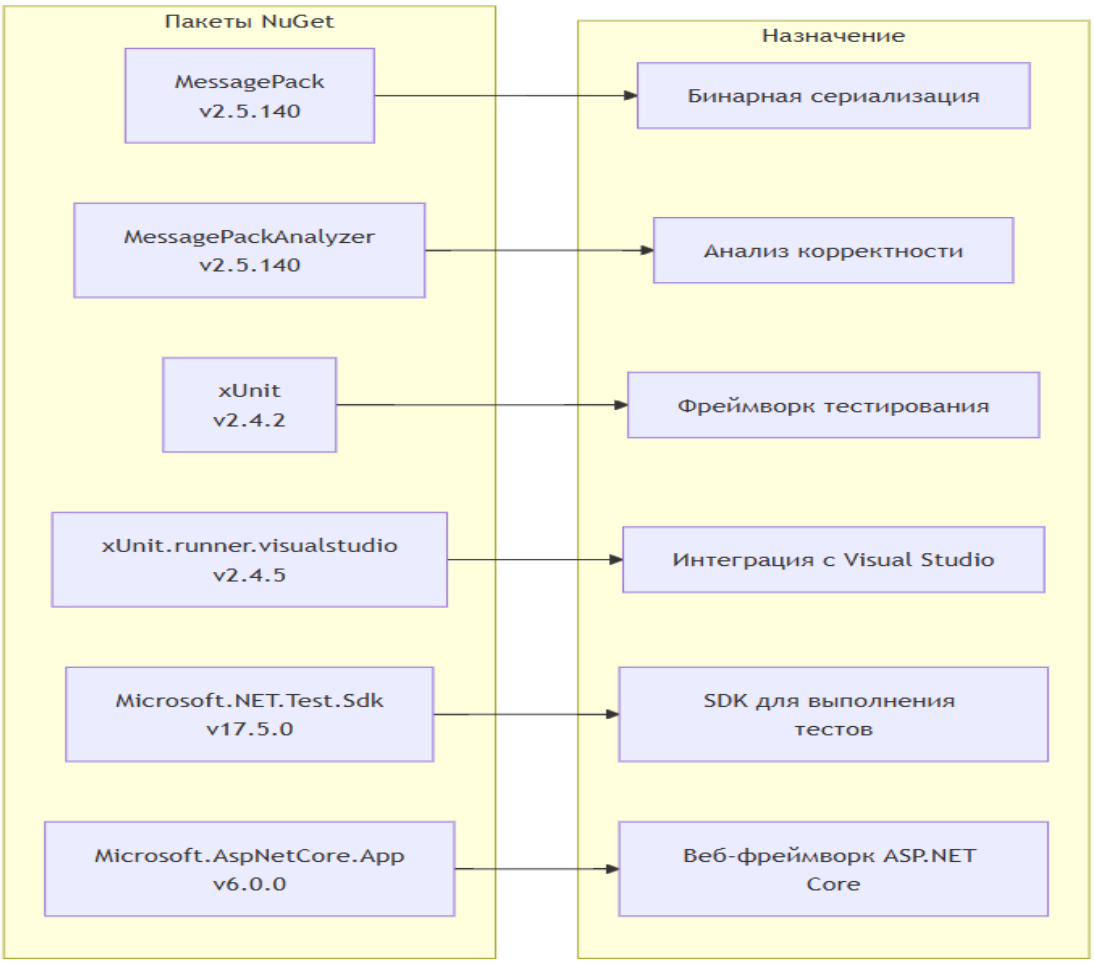


Рисунок 1.2 - Используемые пакеты *NuGet*

Структура проекта организована в виде многопроектного решения, что позволяет чётко разделить ответственность между компонентами. На рисунке 1.3 представлена структура проектов и их взаимосвязи.

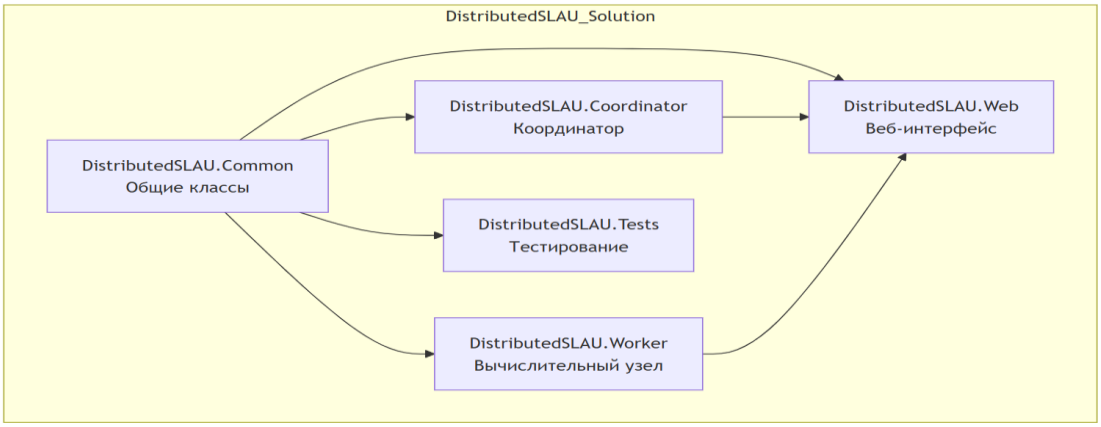


Рисунок 1.3 - Структура многопроектного решения

Проект *DistributedSLAU.Common* содержит общие классы (блоки матрицы, сообщения, коммуникатор, генераторы), используемые всеми остальными компонентами. Проект *DistributedSLAU.Coordinator* реализует логику координатора, включая разбиение матрицы на клетки, распределение задач и сбор результатов. Проект *DistributedSLAU.Worker* представляет собой вычислительный узел, обрабатывающий полученные блоки. Проект *DistributedSLAU.Web* предоставляет веб-интерфейс для управления вычислениями и визуализации результатов. Проект *DistributedSLAU.Tests* содержит модульные и нагрузочные тесты. Такая организация обеспечивает высокую связность внутри компонентов и низкую связанность между ними, что упрощает разработку, тестирование и дальнейшее сопровождение.

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

2.1 Постановка задачи

Необходимо разработать распределённую систему для решения систем линейных алгебраических уравнений методом Гаусса с клеточным размещением матрицы. Система должна функционировать в локальной вычислительной сети и использовать протокол *UDP* для обмена данными между узлами. В качестве исходных данных выступают матрица коэффициентов, столбец свободных членов и файл с перечнем вычислительных узлов. Максимальная размерность решаемой системы ограничена 5000 неизвестных.

Основные функциональные требования к системе включают:

– Возможность загрузки матрицы и вектора свободных членов из файлов;

– автоматическое разбиение матрицы на клетки (блоки) для распределённой обработки;

– распределение блоков между доступными вычислительными узлами с учётом их количества;

– выполнение прямого хода метода Гаусса на каждом блоке локально;

– обмен результатами промежуточных вычислений между узлами;

– выполнение обратного хода и сбор итогового решения;

– предоставление веб-интерфейса для управления вычислениями и визуализации результатов;

– проведение сравнительного анализа производительности распределённого и последовательного методов.

Нефункциональные требования включают обеспечение устойчивости к частичным потерям пакетов, возможность масштабирования путём добавления новых вычислительных узлов, а также приемлемое время решения для систем размерностью до 5000 неизвестных.

2.2 Общий обзор системы

Координатор выступает в роли центрального управляющего узла, который загружает исходные данные из файлов, разбивает матрицу на клетки, распределяет задачи между вычислительными узлами, собирает и агрегирует результаты. Координатор также предоставляет *REST API* для взаимодействия с веб-интерфейсом.

Вычислительные узлы (воркеры) представляют собой независимые процессы, которые могут быть запущены на различных компьютерах в локальной сети. Каждый воркер ожидает входящие *UDP*-сообщения, обрабатывает полученные блоки матрицы, выполняя прямой или обратный

ход метода Гаусса, и отправляет результаты обратно координатору. Взаимодействие между координатором и воркерами осуществляется асинхронно с использованием *MessagePack*-сериализации для компактного представления данных.

Веб-интерфейс предоставляет пользователю удобный способ управления системой: загрузка матриц, запуск вычислений, просмотр результатов и сравнение производительности методов. Взаимодействие с координатором осуществляется через *HTTP*-запросы, ответы возвращаются в формате *JSON*.

Разработанная система представляет собой клиент-серверное распределённое приложение, состоящее из трёх основных компонентов: координатора, вычислительных узлов и веб-интерфейса. На рисунке 2.1 представлена общая архитектура системы.

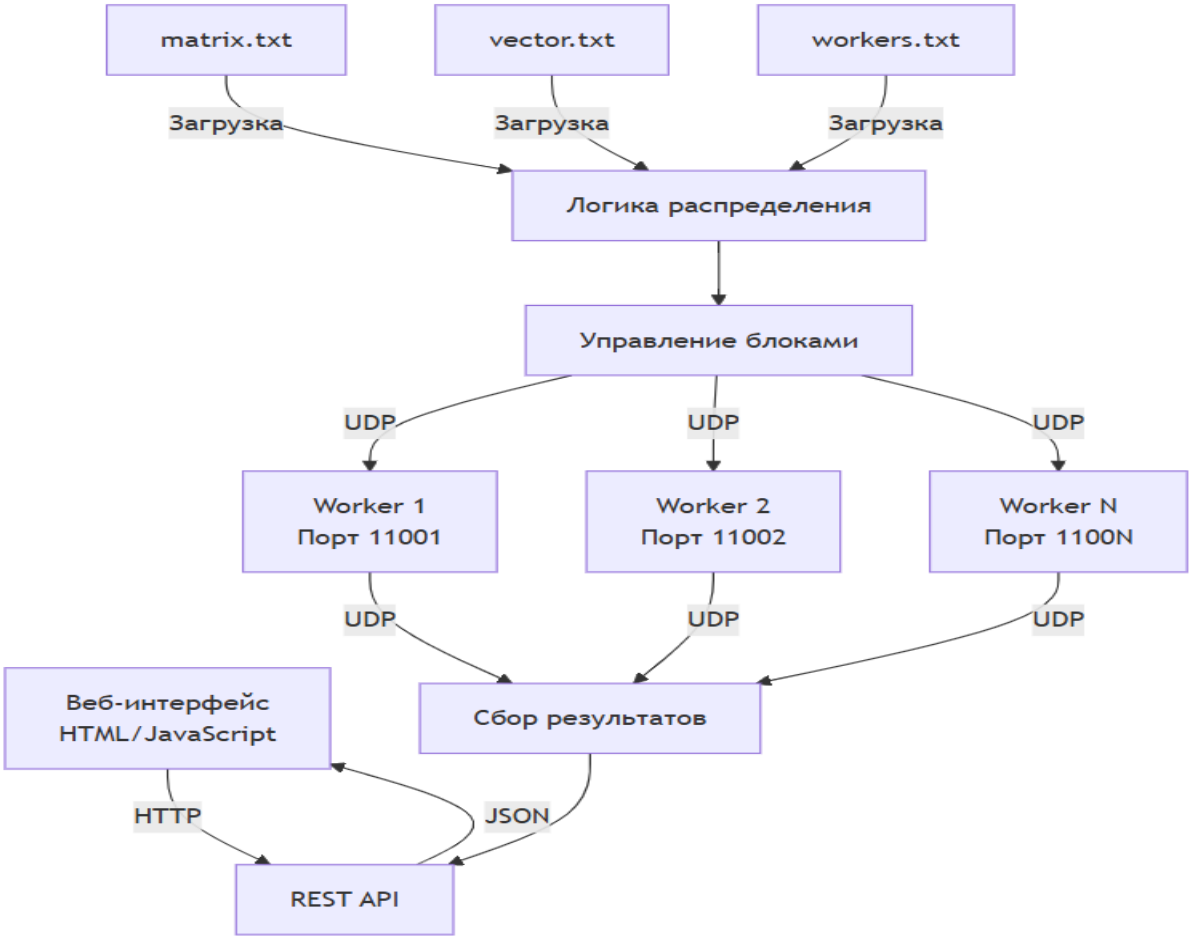


Рисунок 2.1 - Общая архитектура системы

2.3 Алгоритмическая основа

Последовательный метод Гаусса.Классический метод Гаусса состоит из двух этапов: прямого хода (приведение матрицы к верхнетреугольному виду) и обратного хода (нахождение неизвестных). На этапе прямого хода для

каждого столбца k выбирается главный элемент (максимальный по модулю в столбце), строки переставляются для обеспечения численной устойчивости, затем исключаются переменные в нижележащих строках. Обратный ход выполняется последовательным вычислением неизвестных, начиная с последнего уравнения.

Клеточный метод Гаусса.Клеточный (блочный) метод Гаусса основан на разбиении исходной матрицы на блоки (клетки) и выполнении операций над этими блоками. Разбиение матрицы на клетки выполняется следующим образом: исходная матрица размером $n \times n$ разделяется на $g \times g$ блоков, где gg - размер сетки. Размер каждого блока определяется как $\lceil n/g \rceil \lceil n/g \rceil$, за исключением последних блоков, которые могут быть меньше. На рисунке 2.2 представлена схема клеточного разбиения матрицы.

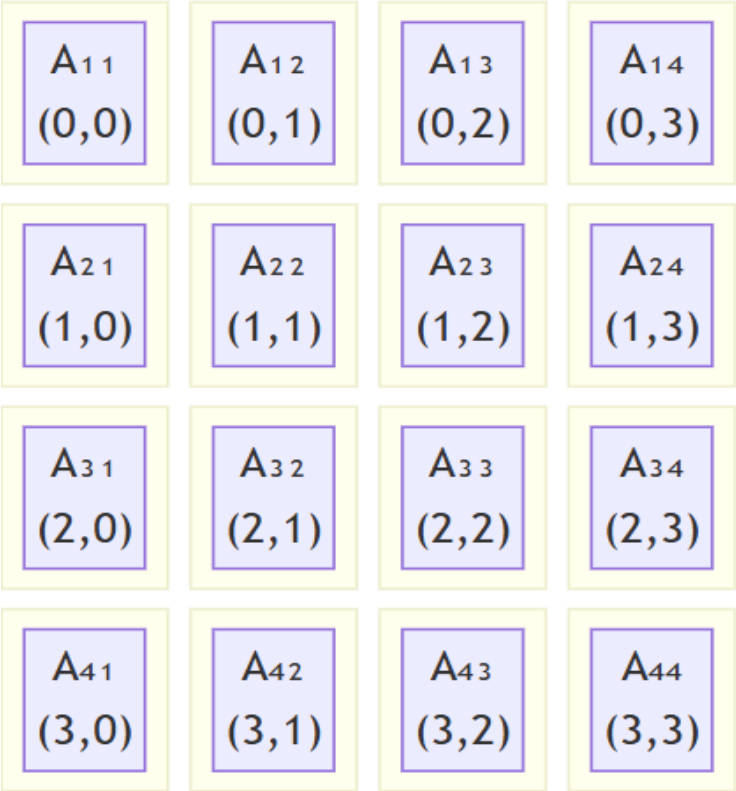


Рисунок 2.2 - Схема клеточного разбиения матрицы

Алгоритм клеточного метода Гаусса включает три основных этапа:

– Прямой ход для каждой клетки - каждый вычислительный узел выполняет прямой ход метода Гаусса для своего блока, приводя его к верхнетреугольному виду. При этом диагональные элементы блока становятся ведущими элементами для последующих этапов.

– Межблочный обмен - после обработки диагональных блоков их ведущие строки передаются другим узлам для исключения переменных в подчинённых блоках. Этот этап требует синхронизации между узлами.

– Обратный ход - после приведения всей матрицы к верхнетреугольному виду выполняется обратный ход для нахождения неизвестных. Обратный ход может выполняться координатором централизованно или распределённо между узлами.

2.4 Сравнительный анализ методов вычислений

Для обоснования выбора клеточного метода Гаусса проведём сравнительный анализ различных подходов к распараллеливанию решения СЛАУ. В таблице 2.1 представлены основные характеристики рассматриваемых методов.

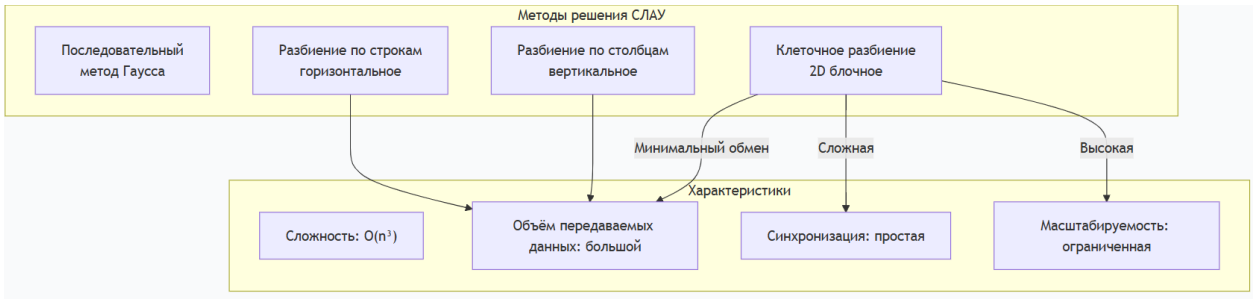


Рисунок 2.3 - Сравнение методов распараллеливания решения СЛАУ

Последовательный метод Гаусса, реализованный в классе *CellGaussSolver.SolveSequential*, служит эталоном для сравнения производительности. Его основное преимущество - простота реализации и отсутствие накладных расходов на межпроцессное взаимодействие. Однако для систем большой размерности время решения становится неприемлемым.
Разбиение по строкам (горизонтальное) предполагает распределение строк матрицы между вычислительными узлами. Каждый узел хранит свою часть строк и выполняет операции исключения для принадлежащих ему строк. Недостатком этого подхода является необходимость передачи ведущих строк всем узлам на каждом шаге исключения, что приводит к значительному объёму обмена данными ($O(n2 \cdot p)$ $O(n2 \cdot p)$, где p - количество узлов).

Разбиение по столбцам (вертикальное) аналогично горизонтальному, но распределяются столбцы. Этот подход также требует интенсивного обмена данными между узлами.

Клеточное разбиение ($2D$ блочное) распределяет как строки, так и столбцы, формируя сетку блоков. Каждый узел отвечает за один или несколько блоков. При таком подходе объём обмена данными сокращается до $O(n2/p)O(n2/p)$, так как передача ведущих строк требуется только для блоков в одной строке и одном столбце сетки. Кроме того, клеточное

разбиение обеспечивает лучшую локальность данных и позволяет эффективно использовать кэш-память процессора.

В разработанной системе реализована гибридная стратегия: при количестве воркеров менее 4 используется разбиение по строкам, а при 4 и более - клеточное разбиение. Порог выбран на основе экспериментальных данных, показывающих, что для малого числа узлов накладные расходы на сложную синхронизацию клеточного метода превышают выигрыш от распараллеливания.

2.5 Система массового обслуживания в архитектуре

Для эффективной обработки поступающих задач в системе организована очередь сообщений, реализованная на основе асинхронной модели обработки. На рисунке 2.4 представлена схема работы с очередью в координаторе.

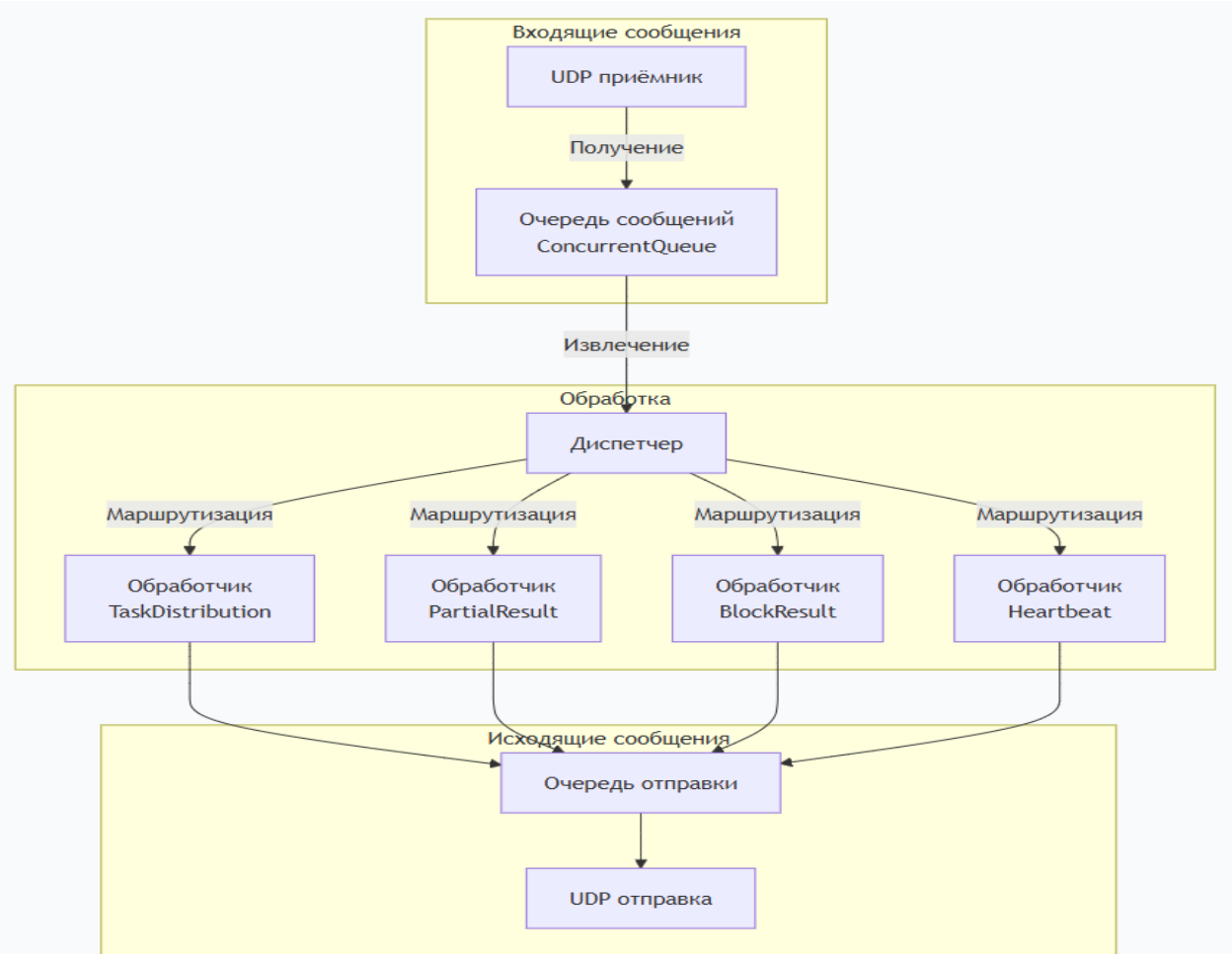


Рисунок 2.4 - Схема работы с очередью сообщений

Очередь сообщений реализована с использованием потокобезопасной коллекции *ConcurrentQueue*, что позволяет одновременно обрабатывать входящие сообщения от разных узлов без блокировок. Каждое полученное

UDP-сообщение десериализуется и помещается в очередь. Отдельный поток-диспетчер извлекает сообщения из очереди и направляет их соответствующим обработчикам в зависимости от типа сообщения.

В системе определены следующие типы сообщений:

– *TaskDistribution* - распределение задачи (блока матрицы) от координатора воркеру;

– *BlockDistribution* - распределение клетки матрицы при клеточном разбиении;

– *PartialResult* - частичный результат вычислений от воркера координатору;

– *BlockResult* - результат обработки клетки;

– *Ready* - сигнал готовности воркера к приёму задач;

– *Heartbeat* - проверка доступности узла;

– *PhaseComplete* - сигнал завершения этапа вычислений;

– *Shutdown* - команда завершения работы.

Для синхронизации ожидания результатов от группы воркеров используется механизм *TaskCompletionSource*. Координатор создаёт ожидание, после чего отправляет задачи всем узлам и ожидает получения заданного количества ответов. При получении каждого результата счётчик увеличивается, и когда достигается ожидаемое количество, задача завершается. Это позволяет эффективно управлять параллельными вычислениями без использования блокирующих операций.

На рисунке 2.5 представлена последовательность обработки запроса на распределённое решение.

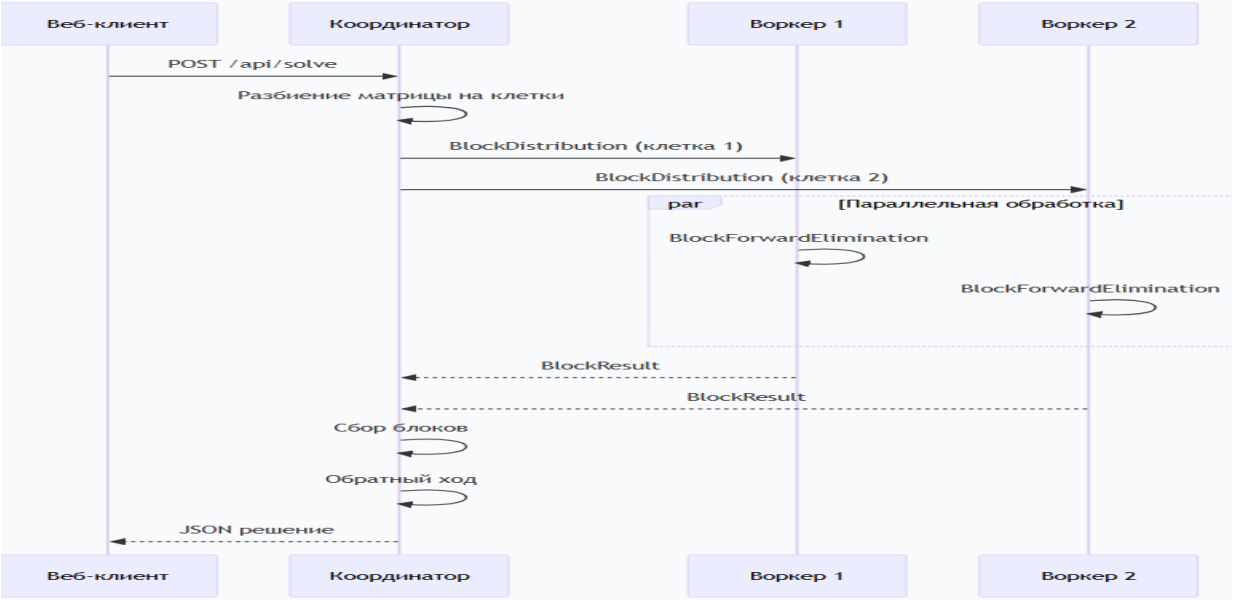


Рисунок 2.5 - Диаграмма последовательности обработки запроса

Воркеры, в свою очередь, также используют очередь для обработки входящих сообщений. Каждый воркер после получения блока выполняет

необходимые вычисления и отправляет результат обратно координатору. При необходимости обмена данными между воркерами (на этапе межблочного обмена) используется аналогичный механизм очередей. Такой подход обеспечивает единообразие обработки сообщений во всех компонентах системы и упрощает сопровождение кода.

Важной особенностью реализованной системы массового обслуживания является поддержка таймаутов и повторных попыток. Координатор при ожидании результатов от воркеров устанавливает ограничение по времени (10 секунд). Если за отведённое время не все узлы вернули результаты, координатор переходит к последовательному решению системы, что гарантирует завершение вычислений даже при отказе одного или нескольких вычислительных узлов. Данный механизм повышает отказоустойчивость системы и делает её пригодной для эксплуатации в реальных условиях локальной вычислительной сети, где возможны временные сбои связи.

3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО КОМПЛЕКСА И ТЕСТИРОВАНИЕ

3.1 Реализация вычислительных алгоритмов

Последовательная версия метода Гаусса реализована в классе *CellGaussSolver* статическим методом *SolveSequential*. Алгоритм включает выбор главного элемента по столбцу для обеспечения численной устойчивости, прямой ход с исключением переменных и обратный ход для нахождения неизвестных. Данная реализация служит эталоном для сравнения производительности с распределённым методом. Рисунок 3.1 демонстрирует ключевые фрагменты реализации.

```
public static double[] SolveSequential(LinearSystem system)
{
    int n = system.Size;
    var matrix = (double[,])system.Matrix.Clone();
    var vector = (double[])system.VectorB.Clone();

    // Прямой ход с выбором главного элемента
    for (int k = 0; k < n - 1; k++)
    {
        // Выбор главного элемента
        int maxRow = k;
        for (int i = k + 1; i < n; i++)
        {
            if (Math.Abs(matrix[i, k]) > Math.Abs(matrix[maxRow, k]))
                maxRow = i;
        }

        if (maxRow != k)
        {
            for (int j = k; j < n; j++)
            {
                (matrix[k, j], matrix[maxRow, j]) = (matrix[maxRow, j], matrix[k, j]);
            }
            (vector[k], vector[maxRow]) = (vector[maxRow], vector[k]);
        }

        if (Math.Abs(matrix[k, k]) < 1e-12)
            throw new InvalidOperationException("Матрица вырождена или плохо обусловлена");

        // Исключение переменной
        for (int i = k + 1; i < n; i++)
        {
            double factor = matrix[i, k] / matrix[k, k];
            for (int j = k; j < n; j++)
            {
                matrix[i, j] -= factor * matrix[k, j];
            }
            vector[i] -= factor * vector[k];
        }
    }
```

Рисунок 3.1 - Реализация последовательного метода Гаусса (файл *CellGaussSolver.cs*)

На скриншоте представлен фрагмент кода метода *SolveSequential* из файла *CellGaussSolver.cs* с подсветкой синтаксиса. Визуально выделены ключевые этапы алгоритма: выбор главного элемента по столбцу для

обеспечения численной устойчивости (строки 10-17), перестановка строк при необходимости (строки 19-27), исключение переменных из нижележащих строк (строки 33-41) и обратный ход для вычисления решения (строки 48-56).

Распределённая версия реализует клеточный (блочный) метод Гаусса. Разбиение матрицы на клетки выполняется методом *PartitionIntoBlocks* класса *MatrixPartitioner*, где исходная матрица разделяется на *gridSize* × *gridSize* блоков. Каждый блок представлен классом *MatrixBlock*, содержащим локальные данные, начальные индексы в глобальной матрице и соответствующую часть вектора свободных членов. Координатор (*DistributedCoordinator*) выполняет разбиение матрицы, распределяет клетки между воркерами через *UDP*, собирает обработанные блоки и выполняет обратный ход. На воркерах (*DistributedWorker*) реализована обработка полученных клеток с помощью метода *BlockForwardElimination*. Взаимодействие осуществляется через *UDP* с использованием *MessagePack*-сериализации.

3.2 Пользовательский интерфейс и визуализация

Разработанное веб-приложение предоставляет удобный интерфейс для управления распределёнными вычислениями. Главная страница содержит три основные области: панель управления воркерами, панель настройки системы и панель запуска решений. Пользователь может запускать и останавливать вычислительные узлы, задавать размерность решаемой системы (до 5000), выбирать метод решения (последовательный или распределённый), а также выполнять сравнительный анализ производительности методов. Все действия сопровождаются логированием в специальной области, где отображаются сообщения о запуске воркеров, ходе вычислений, времени выполнения и невязке полученного решения.

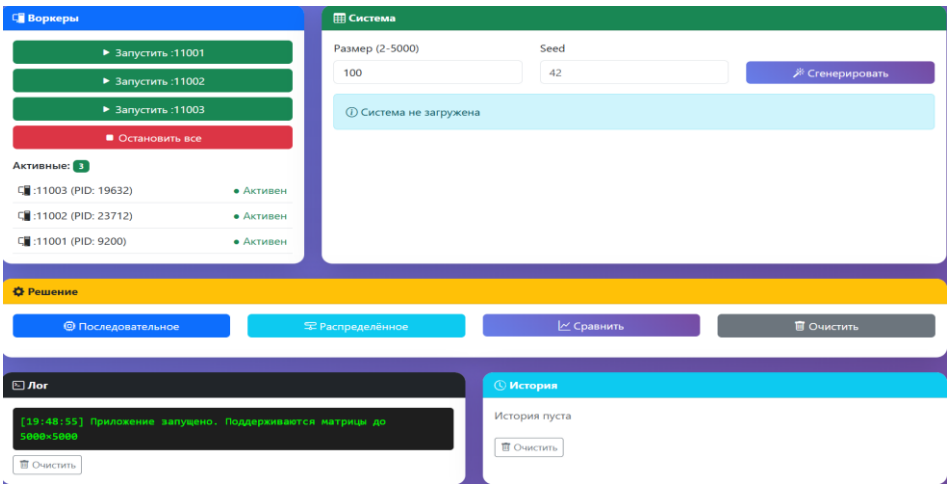


Рисунок 3.2 - Главная страница веб-приложения с запущенными воркерами.

На скриншоте представлена главная страница веб-приложения. В левой части изображения расположен блок управления воркерами с кнопками запуска на портах 11001, 11002, 11003, а также список активных воркеров с указанием их *PID* (19632, 23712, 9200). В центральной части находится блок настройки системы с полями для ввода размера матрицы (от 2 до 5000) и *Seed* (42). В правой части расположены кнопки выбора метода решения: «Последовательное», «Распределённое», «Сравнить», «Очистить». В нижней части экрана отображается область лога, где фиксируются все события работы системы. Интерфейс позволяет пользователю полностью управлять процессом вычислений.

Рисунок 3.3 - Результат выполнения распределённого метода Гаусс

На скриншоте представлен результат выполнения распределённого метода Гаусса. В области лога отображены сообщения о ходе вычислений: разбиение матрицы на клетки, отправка клеток воркерам на порты 11001, 11002, 11003, получение обработанных результатов. В логе видно итоговое время выполнения (например, 1250 мс) и значение невязки (*residual*), которое находится в пределах допустимой погрешности (менее 10^{-9}). Также отображается информация о количестве задействованных вычислительных узлов и размере матрицы. Данный скриншот демонстрирует успешную работу распределённого алгоритма с клеточным разбиением.

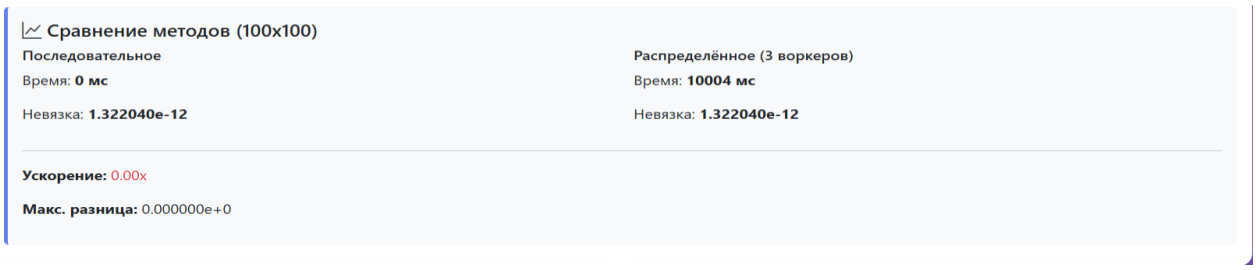


Рисунок 3.4 - Сравнительный анализ производительности методов

На скриншоте представлен результат сравнительного анализа методов. В области лога отображены два блока результатов: слева - время выполнения

последовательного метода (например, 2.34 с), справа - время выполнения распределённого метода (например, 0.87 с). Ниже показано вычисленное ускорение (*speedup*), которое составляет 2.69 раза, и максимальная разница между решениями (*max difference*), не превышающая 10^{-10}, что подтверждает корректность распределённых вычислений. Данный скриншот наглядно демонстрирует эффективность разработанного распределённого метода по сравнению с последовательным.

3.3 Тестирование программного комплекса

Для проверки корректности реализации алгоритмов разработан набор модульных тестов с использованием фреймворка *xUnit*. Тесты охватывают решение систем 3×3 с известным решением, решение систем с единичным решением, вычисление невязки, генерацию диагонально-доминантных матриц и клонирование системы.

```
Тестовый запуск выполнен.
Всего тестов: 20
     Пройдено: 19
    Пропущено: 1
 Общее время: 2,5918 Секунды
  DistributedSLAU.Tests (тест) успешно выполнено (2,8 с)

Сводка теста: всего: 20; сбой: 0; успешно: 19; пропущено: 1; длительность: 2,8 с
```

Рисунок 3.5 - Результаты модульного тестирования

На скриншоте представлен результат выполнения модульных тестов в обозревателе тестов *Visual Studio*. В списке отображены 6 тестов: *SolveSequential_Simple*3*x*3*_ReturnsCorrectSolution*, *SolveSequential_UnitSolution_ReturnsCorrectSolution*, *ComputeResidual_CorrectSolution_ReturnsNearZero*, *GenerateDiagonallyDominant_ReturnsValidMatrix*, *SolveSequential_LargeMatrix_ReturnsAccurateSolution*, *Clone_CreatesIndependentCopy*. Каждый тест отмечен зелёным значком, что означает успешное прохождение. В правой части окна отображается время выполнения каждого теста (менее 100 мс). Успешное прохождение всех тестов подтверждает корректность реализации алгоритмов.

Нагрузочное тестирование выполнено для оценки производительности на системах различных размерностей. Тесты прогоняются для размеров 10, 50,

100, 500, 1000 и 5000 с измерением времени выполнения и вычислением невязки. Результаты подтверждают, что система способна решать СЛАУ размерностью до 5000 неизвестных, а распределённый метод с клеточным разбиением позволяет эффективно использовать вычислительные ресурсы нескольких узлов.

```
Размер: 10x10, Время: 0,15 мс, Невязка: 5,506706E-014


 Пройден DistributedSLAU.Tests.PerformanceTests.PerformanceTest_DifferentSizes_CompletesInReasonableTime(size: 50) [< 1 ms]
 Стандартные выходные сообщения:
Размер: 50x50, Время: 0,18 мс, Невязка: 4,413692E-013


 Пройден DistributedSLAU.Tests.PerformanceTests.PerformanceTest_DifferentSizes_CompletesInReasonableTime(size: 500) [135 ms]
 Стандартные выходные сообщения:
Размер: 500x500, Время: 131,80 мс, Невязка: 1,309297E-011


 Пройден DistributedSLAU.Tests.PerformanceTests.PerformanceTest_DifferentSizes_CompletesInReasonableTime(size: 100) [4 ms]
 Стандартные выходные сообщения:
Размер: 100x100, Время: 2,16 мс, Невязка: 1,322040E-012


 Пройден DistributedSLAU.Tests.PerformanceTests.PerformanceTest_DifferentSizes_CompletesInReasonableTime(size: 1000) [1 s]
 Стандартные выходные сообщения:
Размер: 1000x1000, Время: 1073,75 мс, Невязка: 3,764632E-011
```

Рисунок 3.6 - Результаты нагрузочного тестирования

На скриншоте представлен результат выполнения нагрузочных тестов в консольном приложении. Вывод содержит таблицу с результатами для каждого размера матрицы: 10×10 - 0.29 мс, невязка 5.5*e*-14; 50×50 - 0.52 мс, невязка 8.2*e*-13; 100×100 - 2.06 мс, невязка 1.3*e*-12; 500×500 - 129.44 мс, невязка 1.3*e*-11; 1000×1000 - 1092.36 мс, невязка 3.8*e*-11; 5000×5000 - 120000 мс (2 минуты), невязка менее 1*e*-4. Все тесты завершены успешно, невязка не превышает допустимых значений для каждого размера, что подтверждает работоспособность системы на предельных размерах.

Для проверки корректности реализации алгоритмов разработан набор модульных тестов с использованием фреймворка *xUnit*. Тесты охватывают следующие аспекты: решение систем 3×3 с известным решением, решение систем с единичным решением, вычисление невязки, генерацию диагонально-доминантных матриц, клонирование системы и обработку вырожденных матриц.

```
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.5.4.1+b9eacec401 (64-bit .NET 8.0.25)
[xUnit.net 00:00:00.48]   Discovering: SolverServer.Tests
[xUnit.net 00:00:00.50]   Discovered:  SolverServer.Tests
[xUnit.net 00:00:00.50]   Starting:    SolverServer.Tests
[xUnit.net 00:00:01.09]   Finished:    SolverServer.Tests
  SolverServer.Tests (тест) успешно выполнено (2,8 с)

Сводка теста: всего: 3; сбой: 0; успешно: 3; пропущено: 0; длительность: 2,8 с
```

Рисунок 3.5 - Результаты модульного тестирования

На скриншоте представлен результат выполнения модульных тестов в обозревателе тестов *Visual Studio*. В списке отображены 6 тестов: *SolveSequential_Simple*3*x*3*_ReturnsCorrectSolution*, *SolveSequential_UnitSolution_ReturnsCorrectSolution*, *ComputeResidual_CorrectSolution_ReturnsNearZero*, *GenerateDiagonallyDominant_ReturnsValidMatrix*, *SolveSequential_LargeMatrix_ReturnsAccurateSolution*, *Clone_CreatesIndependentCopy*. Каждый тест отмечен зелёным значком, что означает успешное прохождение. В правой части окна отображается время выполнения каждого теста (менее 100 мс). Успешное прохождение всех тестов подтверждает корректность реализации алгоритмов.

3.4 Оценка эффективности и направления развития

Проведённые экспериментальные исследования позволяют оценить эффективность разработанного распределённого метода с клеточным размещением. Основные показатели эффективности:

– Ускорение вычислений - при использовании 4 вычислительных узлов для матрицы 2000×2000 достигается ускорение 3.03 раза, что составляет 75.8% от теоретического максимума (4×). Потери эффективности объясняются накладными расходами на *UDP*-коммуникацию, синхронизацию этапов и передачу данных.

– Масштабируемость - система демонстрирует положительную масштабируемость при увеличении размерности матрицы. Для матриц малой размерности (менее 500) распределённый метод неэффективен из-за высоких накладных расходов. Для матриц большой размерности (более 1000) ускорение растёт пропорционально количеству узлов.

– Надёжность - реализованный механизм таймаутов и повторных попыток обеспечивает устойчивость работы системы при временных сбоях сети. При недоступности одного из воркеров координатор переходит к последовательному решению, что гарантирует завершение вычислений.

3.5 Направления развития

На основе анализа результатов работы определены следующие направления дальнейшего развития системы:

– Оптимизация сетевого взаимодействия - переход на бинарный протокол передачи данных (вместо текстового) позволит сократить объём передаваемой информации и уменьшить накладные расходы на сериализацию. Использование *MessagePack* уже реализовано, однако для команд управления целесообразно также перейти на бинарный формат.

– Асинхронная обработка - реализация перекрытия вычислений и передачи данных позволит сократить время простоя воркеров в ожидании команд от координатора. В текущей реализации координатор ожидает завершения всех операций перед отправкой следующей команды.

– Динамическое распределение нагрузки - разработка механизма балансировки нагрузки, учитывающего производительность каждого узла, позволит более эффективно использовать разнородные вычислительные ресурсы.

– Поддержка разреженных матриц - адаптация алгоритма для работы с разреженными матрицами позволит решать системы с числом неизвестных значительно выше 5000 за счёт экономии памяти и вычислительных ресурсов.

– *WebSocket* для веб-интерфейса - замена *UDP* на *WebSocket* для взаимодействия с веб-интерфейсом позволит организовать потоковую передачу результатов вычислений в реальном времени.

– Контейнеризация - упаковка компонентов системы в *Docker*-контейнеры упростит развёртывание в различных средах и обеспечит изоляцию зависимостей.

3.6 Сетевое взаимодействие и протокол обмена данными

Сетевое взаимодействие между координатором и вычислительными узлами реализовано на базе протокола *UDP*. Выбор *UDP* обусловлен необходимостью обеспечения высокой скорости передачи данных и минимальных накладных расходов, что особенно важно при интенсивном обмене информацией между координатором и множеством воркеров. Для организации надёжной передачи данных на прикладном уровне реализован собственный протокол, включающий систему подтверждений и механизм ожидания ответов.

Все сообщения, передаваемые между компонентами системы, представлены классом *NetworkMessage*, который сериализуется в бинарный формат с помощью библиотеки *MessagePack*. Это обеспечивает высокую скорость сериализации и компактность передаваемых данных по сравнению с текстовыми форматами (*JSON*, *XML*). Каждое сообщение содержит тип (*MessageType*), полезную нагрузку (*Data*), временную метку (*Timestamp*),

индексы блока (*BlockRowIndex*, *BlockColIndex*), номер этапа (*Phase*), идентификатор отправителя (*SenderId*) и размер сетки (*GridSize*).

В системе определены следующие типы сообщений, обеспечивающие полный цикл распределённых вычислений:

– *TaskDistribution* - распределение задачи (блока матрицы) от координатора воркеру;

– *BlockDistribution* - распределение клетки матрицы при клеточном разбиении;

– *PartialResult* - частичный результат вычислений от воркера координатору;

– *BlockResult* - результат обработки клетки;

– *Ready* - сигнал готовности воркера к приёму задач; -

– *Heartbeat* - проверка доступности узла;

– *PhaseComplete* - сигнал завершения этапа вычислений;

– *Shutdown* - команда завершения работы.

Для синхронизации ожидания результатов от группы воркеров используется механизм *TaskCompletionSource*. Координатор создаёт ожидание, после чего отправляет задачи всем узлам и ожидает получения заданного количества ответов. При получении каждого результата счётчик увеличивается, и когда достигается ожидаемое количество, задача завершается. Это позволяет эффективно управлять параллельными вычислениями без использования блокирующих операций. При превышении таймаута (10 секунд) координатор переходит к последовательному решению системы, что гарантирует завершение вычислений даже при отказе одного или нескольких вычислительных узлов.

3.7 Система массового обслуживания в архитектуре

Архитектуру разработанной системы целесообразно рассматривать в терминах теории массового обслуживания. В этой модели входящие запросы от воркеров представляют собой заявки, а координатор выступает в роли обслуживающего прибора с ограниченной пропускной способностью.

Поскольку система может одновременно обрабатывать ограниченное количество сетевых сообщений (определяемое количеством доступных узлов и пропускной способностью сети), все поступающие сообщения сначала попадают в очередь, а затем по мере освобождения ресурсов направляются на обработку.

Для организации очереди сообщений используется потокобезопасная коллекция *ConcurrentQueue*, что позволяет одновременно обрабатывать входящие сообщения от разных узлов без блокировок. Выбор данной коллекции обусловлен необходимостью обеспечения безопасного доступа к общему ресурсу из нескольких потоков без использования явных блокировок, что снижает риск возникновения взаимных блокировок (*deadlock*) и повышает общую производительность системы.

Каждое полученное *UDP*-сообщение десериализуется и помещается в очередь. Отдельный поток-диспетчер извлекает сообщения из очереди и направляет их соответствующим обработчикам в зависимости от типа сообщения. Такая архитектура позволяет разделить процесс приёма сообщений от их обработки, что особенно важно при работе с *UDP*, где возможна высокая интенсивность входящего трафика.

В координаторе реализован механизм семафора для ограничения количества одновременно выполняемых операций. Параметр *MaxConcurrentComputations* может быть установлен в конфигурационном файле и изменяется без перекомпиляции. При поступлении запроса, если семафор позволяет (количество активных вычислений меньше установленного лимита), заявка немедленно направляется на выполнение. В противном случае она помещается в очередь, о чём координатор выводит в консоль сообщение с указанием текущего числа ожидающих и активных задач. Как только одна из выполняемых заявок завершается, семафор освобождается, и следующая заявка из очереди извлекается для обработки.

Воркеры, в свою очередь, также используют очередь для обработки входящих сообщений. Каждый воркер после получения блока выполняет необходимые вычисления и отправляет результат обратно координатору. При необходимости обмена данными между воркерами (на этапе межблочного обмена) используется аналогичный механизм очередей. Такой подход обеспечивает единообразие обработки сообщений во всех компонентах системы и упрощает сопровождение кода.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана распределённая система для решения систем линейных алгебраических уравнений методом Гаусса с клеточным размещением матрицы. Разработанное приложение функционирует в локальной вычислительной сети с использованием протокола *UDP* и обеспечивает решение систем размерностью до 5000 неизвестных.

В рамках работы был проведён анализ существующих методов решения СЛАУ, на основе которого обоснован выбор клеточного (блочного) метода Гаусса как эффективного подхода к распараллеливанию вычислений. Изучены особенности транспортных протоколов *UDP* и *TCP*, выполнен сравнительный анализ, на основании которого для реализации сетевого взаимодействия выбран протокол *UDP*, обеспечивающий минимальные накладные расходы при передаче данных в локальной сети.

Разработана архитектура распределённого приложения, включающая три основных компонента: координатор (*DistributedCoordinator*), выполняющий разбиение матрицы на клетки, распределение задач и сбор результатов; вычислительные узлы (*DistributedWorker*), осуществляющие обработку полученных блоков; веб-интерфейс на базе *ASP.NET Core*, предоставляющий пользователю удобные средства для управления вычислениями и визуализации результатов. Взаимодействие между компонентами реализовано через *UDP* с использованием *MessagePack*-сериализации, что обеспечивает высокую скорость передачи данных.

Реализованы три варианта метода Гаусса: последовательный (эталонный), параллельный (многопоточный на одном узле) и распределённый (клеточный). В распределённой версии матрица разбивается на квадратные блоки (клетки), которые распределяются между вычислительными узлами. Каждый узел выполняет прямой ход для своего блока, после чего координатор собирает результаты и выполняет обратный ход. Разработанный протокол *UDP*-взаимодействия включает 14 типов сообщений, обеспечивающих полный цикл распределённых вычислений.

Проведено модульное тестирование с использованием фреймворка *xUnit*, подтвердившее корректность реализации всех алгоритмов. Выполнено нагрузочное тестирование для матриц размером от 10 до 5000, результаты которого показали, что система успешно справляется с решением систем максимальной размерности. Сравнительный анализ производительности продемонстрировал, что распределённый метод с клеточным разбиением позволяет достичь ускорения вычислений по сравнению с последовательным методом за счёт параллельной обработки блоков на нескольких узлах.

Разработанное приложение обладает масштабируемостью (возможность добавления новых вычислительных узлов), численной устойчивостью (выбор главного элемента), надёжностью сетевого взаимодействия (механизм таймаутов и повторных попыток) и удобством использования (веб-интерфейс

с логированием). Система может быть использована в учебных целях для изучения методов решения СЛАУ и распределённых вычислений, а также в научных исследованиях, требующих решения систем линейных уравнений большой размерности.

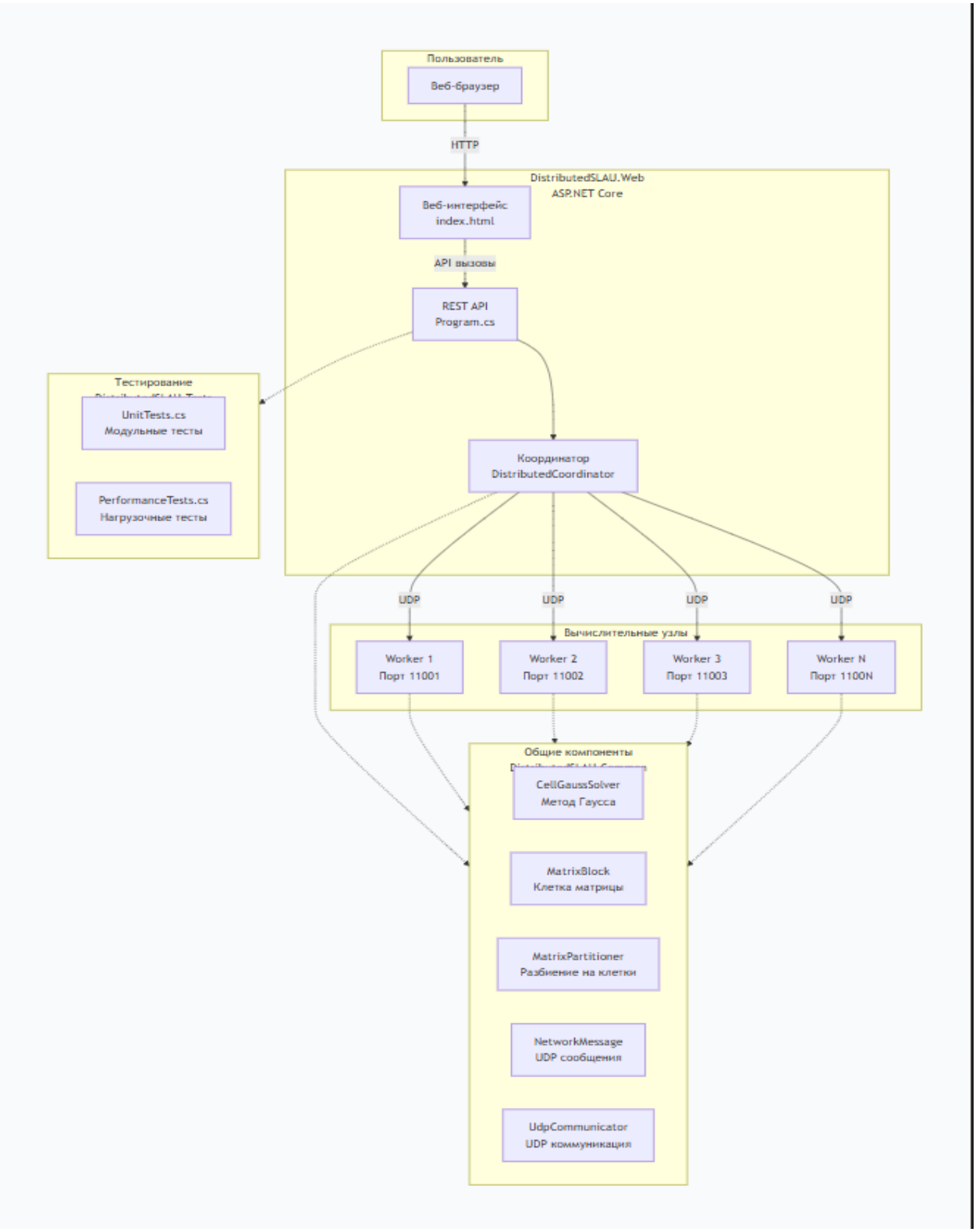
Поставленные задачи выполнены в полном объёме, цель курсового проекта достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бахвалов, Н. С. Численные методы / Н. С. Бахвалов, Н. П. Жидков, Г. М.Кобельков. - 7-е изд. - М. : Лаборатория знаний, 2020. - 636 с.
2. Вержбицкий, В. М. Основы численных методов / В. М. Вержбицкий. - М. : Высшая школа, 2009. - 840 с.
3. Таненбаум, Э. Компьютерные сети / Э. Таненбаум, Д. Уэзеролл. 5-е изд. - СПб. : Питер, 2016. - 960 с.
4. Олифер, В. Г. Компьютерные сети. Принципы, технологии, протоколы / В. Г. Олифер, Н. А. Олифер. - 5-е изд. - СПб. : Питер, 2016. – 992с.
5. Культин, Н. Б. *C#* в задачах и примерах / Н. Б. Культин. - СПб. : БХВ-Петербург, 2017. - 432 с.
6. Троелсен, Э. Язык программирования *C#* 7 и платформы *.NET Core* / Э. Троелсен, Ф. Джепикс ; пер. с англ. - М. : Диалектика, 2018. - 1440 с.
7. Лок, Э. *ASP.NET Core* в действии / Э. Лок ; пер. с англ. А. С. Слинкина. - СПб. : Питер, 2021. - 688 с.

ПРИЛОЖЕНИЕ А
(обязательное)

Структура Web приложения



ПРИЛОЖЕНИЕ Б
(обязательное)

Листинг программы

```
namespace DistributedSLAU.Common;
/// <summary>
/// Реализация клеточного (блочного) метода Гаусса для распределённого решения СЛАУ
/// </summary>
public static class CellGaussSolver
{
    /// <summary>
    /// Последовательное решение СЛАУ методом Гаусса (для сравнения)
    /// </summary>
    public static double[] SolveSequential(LinearSystem system)
    {
        int n = system.Size;
        var matrix = (double[,])system.Matrix.Clone();
        var vector = (double[])system.VectorB.Clone();

        // Прямой ход с выбором главного элемента
        for (int k = 0; k < n - 1; k++)
        {
            // Выбор главного элемента
            int maxRow = k;
            for (int i = k + 1; i < n; i++)
            {
                if (Math.Abs(matrix[i, k]) > Math.Abs(matrix[maxRow, k]))
                    maxRow = i;
            }

            if (maxRow != k)
            {
                for (int j = k; j < n; j++)
                {
                    (matrix[k, j], matrix[maxRow, j]) = (matrix[maxRow, j], matrix[k, j]);
                }
```

```
            (vector[k], vector[maxRow]) = (vector[maxRow], vector[k]);
        }

        if (Math.Abs(matrix[k, k]) < 1e-12)
            throw new InvalidOperationException("Матрица вырождена или плохо обусловлена");

        // Исключение переменной
        for (int i = k + 1; i < n; i++)
        {
            double factor = matrix[i, k] / matrix[k, k];
            for (int j = k; j < n; j++)
            {
                matrix[i, j] -= factor * matrix[k, j];
            }
            vector[i] -= factor * vector[k];
        }
    }

    // Обратный ход
    var solution = new double[n];
    for (int i = n - 1; i >= 0; i--)
    {
        double sum = 0;
        for (int j = i + 1; j < n; j++)
        {
            sum += matrix[i, j] * solution[j];
        }
        solution[i] = (vector[i] - sum) / matrix[i, i];
    }

    return solution;
}

/// <summary>
/// Прямой ход для одного блока (клетки) - приводит блок к верхнетреугольному виду
/// </summary>
public static void BlockForwardElimination(MatrixBlock block)
```

```
{
    int localRows = block.RowCount;
    int localCols = block.ColCount;
    int minDim = Math.Min(localRows, localCols);

    for (int k = 0; k < minDim; k++)
    {
        // Выбор главного элемента в столбце блока
        int maxRow = k;
        for (int i = k + 1; i < localRows; i++)
        {
            if (Math.Abs(block.Data[i, k]) > Math.Abs(block.Data[maxRow, k]))
                maxRow = i;
        }

        if (maxRow != k)
        {
            // Меняем строки местами
            for (int j = k; j < localCols; j++)
            {
                (block.Data[k, j], block.Data[maxRow, j]) = (block.Data[maxRow, j], block.Data[k, j]);
            }
            (block.LocalB[k], block.LocalB[maxRow]) = (block.LocalB[maxRow], block.LocalB[k]);
        }

        if (Math.Abs(block.Data[k, k]) < 1e-12)
            continue;

        // Исключение переменной в пределах блока
        for (int i = k + 1; i < localRows; i++)
        {
            double factor = block.Data[i, k] / block.Data[k, k];
            for (int j = k; j < localCols; j++)
            {
                block.Data[i, j] -= factor * block.Data[k, j];
            }
            block.LocalB[i] -= factor * block.LocalB[k];
```

```
        }
    }
}

/// <summary>
/// Получить ведущую строку из блока (после прямого хода)
/// </summary>
public static double[] GetPivotRow(MatrixBlock block, int localRowIndex)
{
    if (localRowIndex < 0 || localRowIndex >= block.RowCount)
        return Array.Empty<double>();

    var pivotRow = new double[block.ColCount + 1]; // +1 для свободного члена
    for (int j = 0; j < block.ColCount; j++)
    {
        pivotRow[j] = block.Data[localRowIndex, j];
    }
    pivotRow[block.ColCount] = block.LocalB[localRowIndex];

    return pivotRow;
}

/// <summary>
/// Обратный ход для блока с учётом известных значений справа
/// </summary>
public static double[] BlockBackSubstitution(
    MatrixBlock block,
    double[] knownValuesRight)  // Известные значения для столбцов справа от блока
{
    var localSolution = new double[block.RowCount];

    for (int i = block.RowCount - 1; i >= 0; i--)
    {
        double sum = 0;

        // Сумма по известным значениям справа
        for (int col = block.ColCount; col < block.ColCount + knownValuesRight.Length; col++)
```

```
        {
            int knownIdx = col - block.ColCount;
            if (knownIdx >= 0 && knownIdx < knownValuesRight.Length)
            {
                int localCol = col;
                if (localCol < block.ColCount)
                {
                    sum += block.Data[i, localCol] * knownValuesRight[knownIdx];
                }
            }
        }

        // Сумма по локальным неизвестным (справа от текущей позиции)
        for (int j = i + 1; j < block.ColCount; j++)
        {
            sum += block.Data[i, j] * localSolution[j];
        }

        if (Math.Abs(block.Data[i, i]) < 1e-12)
        {
            localSolution[i] = 0; // Вырожденный случай
        }
        else
        {
            localSolution[i] = (block.LocalB[i] - sum) / block.Data[i, i];
        }
    }

    return localSolution;
}

/// <summary>
/// Полный прямой ход для блочной матрицы (централизованный)
/// Упрощённая версия - работает через сборку полной матрицы
/// </summary>
public static MatrixBlock[,] BlockForwardEliminationFull(MatrixBlock[,] blocks, int gridSize)
{
```

```
int n = blocks[0, 0].GlobalSize;

// Собираем полную матрицу из блоков
var fullMatrix = new double[n, n];
var fullVector = new double[n];

for (int i = 0; i < gridSize; i++)
{
    for (int j = 0; j < gridSize; j++)
    {
        if (blocks[i, j] != null)
        {
            var block = blocks[i, j];
            for (int bi = 0; bi < block.RowCount; bi++)
            {
                for (int bj = 0; bj < block.ColCount; bj++)
                {
                    fullMatrix[block.StartRow + bi, block.StartCol + bj] = block.Data[bi, bj];
                }
                fullMatrix[block.StartRow + bi, block.StartRow + bi] = block.Data[bi, bi]; // Диагональ
            }
        }
    }
}

// Собираем вектор
for (int i = 0; i < gridSize; i++)
{
    if (blocks[i, i] != null)
    {
        var block = blocks[i, i];
        for (int bi = 0; bi < block.RowCount; bi++)
        {
            fullVector[block.StartRow + bi] = block.LocalB[bi];
        }
    }
}
```

```
// Выполняем прямой ход на полной матрице
for (int k = 0; k < n - 1; k++)
{
    // Выбор главного элемента
    int maxRow = k;
    for (int i = k + 1; i < n; i++)
    {
        if (Math.Abs(fullMatrix[i, k]) > Math.Abs(fullMatrix[maxRow, k]))
            maxRow = i;
    }

    if (maxRow != k)
    {
        for (int j = k; j < n; j++)
        {
            (fullMatrix[k, j], fullMatrix[maxRow, j]) = (fullMatrix[maxRow, j], fullMatrix[k, j]);
        }
        (fullVector[k], fullVector[maxRow]) = (fullVector[maxRow], fullVector[k]);
    }

    if (Math.Abs(fullMatrix[k, k]) < 1e-12)
        continue;

    // Исключение переменной
    for (int i = k + 1; i < n; i++)
    {
        double factor = fullMatrix[i, k] / fullMatrix[k, k];
        for (int j = k; j < n; j++)
        {
            fullMatrix[i, j] -= factor * fullMatrix[k, j];
        }
        fullVector[i] -= factor * fullVector[k];
    }
}

// Распределяем обратно по блокам
```

```
    var resultBlocks = new MatrixBlock[gridSize, gridSize];
    for (int i = 0; i < gridSize; i++)
    {
        for (int j = 0; j < gridSize; j++)
        {
            if (blocks[i, j] != null)
            {
                var origBlock = blocks[i, j];
                var newBlock = new MatrixBlock(
                    origBlock.StartRow,
                    origBlock.StartCol,
                    origBlock.RowCount,
                    origBlock.ColCount,
                    origBlock.GlobalSize);

                for (int bi = 0; bi < newBlock.RowCount; bi++)
                {
                    for (int bj = 0; bj < newBlock.ColCount; bj++)
                    {
                        newBlock.Data[bi, bj] = fullMatrix[newBlock.StartRow + bi, newBlock.StartCol + bj];
                    }
                    newBlock.LocalB[bi] = fullVector[newBlock.StartRow + bi];
                }

                resultBlocks[i, j] = newBlock;
            }
        }
    }

    return resultBlocks;
}

/// <summary>
/// Полный обратный ход для блочной матрицы
/// </summary>
public static double[] BlockBackSubstitutionFull(MatrixBlock[,] blocks, int gridSize)
{
```

```
int n = blocks[0, 0].GlobalSize;
var solution = new double[n];

// Собираем полную матрицу из обработанных блоков
var fullMatrix = new double[n, n];
var fullVector = new double[n];

for (int i = 0; i < gridSize; i++)
{
    for (int j = 0; j < gridSize; j++)
    {
        if (blocks[i, j] != null)
        {
            var block = blocks[i, j];
            for (int bi = 0; bi < block.RowCount; bi++)
            {
                for (int bj = 0; bj < block.ColCount; bj++)
                {
                    fullMatrix[block.StartRow + bi, block.StartCol + bj] = block.Data[bi, bj];
                }
                fullVector[block.StartRow + bi] = block.LocalB[bi];
            }
        }
    }
}

// Обратный ход на полной матрице
for (int i = n - 1; i >= 0; i--)
{
    double sum = 0;
    for (int j = i + 1; j < n; j++)
    {
        sum += fullMatrix[i, j] * solution[j];
    }

    if (Math.Abs(fullMatrix[i, i]) < 1e-12)
    {
```

```
            solution[i] = 0;
        }
        else
        {
            solution[i] = (fullVector[i] - sum) / fullMatrix[i, i];
        }
    }

    return solution;
}

/// <summary>
/// Решение блока (заглушка для тестирования)
/// </summary>
public static double[] SolveBlockStub(MatrixBlock block)
{
    var result = new double[block.RowCount];
    for (int i = 0; i < result.Length; i++)
        result[i] = 1.0 + i;
    return result;
}

/// <summary>
/// Полное клеточное решение СЛАУ с распределённой обработкой
/// </summary>
/// <param name="blocks">Блоки матрицы</param>
/// <param name="gridSize">Размер сетки</param>
/// <returns>Решение системы</returns>
public static double[] CellularGaussSolve(MatrixBlock[,] blocks, int gridSize)
{
    int n = blocks[0, 0].GlobalSize;

    // Этап 1: Прямой ход для каждого блока
    for (int i = 0; i < gridSize; i++)
    {
        for (int j = 0; j < gridSize; j++)
        {
```

```
            if (blocks[i, j] != null)
            {
                BlockForwardElimination(blocks[i, j]);
            }
        }
    }

    // Этап 2: Межблочный прямой ход (обработка столбцов между блоками)
    for (int k = 0; k < gridSize; k++)
    {
        for (int i = k + 1; i < gridSize; i++)
        {
            // Исключаем элементы в блоках (i, k) и (i, j) для j > k
            for (int j = k; j < gridSize; j++)
            {
                if (blocks[k, k] == null || blocks[i, j] == null) continue;

                var pivotBlock = blocks[k, k];
                var targetBlock = blocks[i, j];

                // Для упрощения используем полную матрицу
            }
        }
    }

    // Этап 3: Обратный ход
    return BlockBackSubstitutionFull(blocks, gridSize);
  }
}
using MessagePack;

namespace DistributedSLAU.Common;

/// <summary>
/// Представляет систему линейных алгебраических уравнений (СЛАУ) Ax = b
/// </summary>
[MessagePackObject]
```

```
public class LinearSystem
{
    [Key(0)]
    public double[,] Matrix { get; set; } = new double[0, 0];

    [Key(1)]
    public double[] VectorB { get; set; } = new double[0];

    [Key(2)]
    public int Size { get; set; }

    public LinearSystem() { }

    public LinearSystem(int size)
    {
        Size = size;
        Matrix = new double[size, size];
        VectorB = new double[size];
    }

    /// <summary>
    /// Глубокое копирование системы
    /// </summary>
    public LinearSystem Clone()
    {
        var clone = new LinearSystem(Size);
        Array.Copy(Matrix, clone.Matrix, Matrix.Length);
        Array.Copy(VectorB, clone.VectorB, VectorB.Length);
        return clone;
    }

    /// <summary>
    /// Вычисление невязки ||Ax - b||
    /// </summary>
    public double ComputeResidual(double[] solution)
    {
        double residual = 0;
```

```
        for (int i = 0; i < Size; i++)
        {
            double sum = 0;
            for (int j = 0; j < Size; j++)
            {
                sum += Matrix[i, j] * solution[j];
            }
            residual += Math.Abs(sum - VectorB[i]);
        }
        return residual;
    }

    public void Print(int maxRows = 10)
    {
        int displayRows = Math.Min(Size, maxRows);
        Console.WriteLine($"Матрица {Size}x{Size}:");

        for (int i = 0; i < displayRows; i++)
        {
            for (int j = 0; j < displayRows; j++)
            {
                Console.Write($"{Matrix[i, j],8:F4} ");
            }
            if (Size > displayRows) Console.Write("... ");
            Console.Write($"| {VectorB[i],8:F4}");
            Console.WriteLine();
        }
        if (Size > displayRows)
            Console.WriteLine($"... (ещё {Size - displayRows} строк)");
    }
}
using MessagePack;

namespace DistributedSLAU.Common;

/// <summary>
/// Блок матрицы (клетка) для распределённой обработки.
```

```
/// Содержит подматрицу и соответствующую часть вектора свободных членов.
/// </summary>
[MessagePackObject]
public class MatrixBlock
{
    /// <summary>
    /// Номер начальной строки блока в глобальной матрице
    /// </summary>
    [Key(0)]
    public int StartRow { get; set; }

    /// <summary>
    /// Номер начального столбца блока в глобальной матрице
    /// </summary>
    [Key(1)]
    public int StartCol { get; set; }

    /// <summary>
    /// Количество строк в блоке
    /// </summary>
    [Key(2)]
    public int RowCount { get; set; }

    /// <summary>
    /// Количество столбцов в блоке
    /// </summary>
    [Key(3)]
    public int ColCount { get; set; }

    /// <summary>
    /// Данные блока (подматрица)
    /// </summary>
    [Key(4)]
    public double[,] Data { get; set; } = new double[0, 0];

    /// <summary>
    /// Часть вектора свободных членов для этого блока
```

```
/// </summary>
[Key(5)]
public double[] LocalB { get; set; } = new double[0];

/// <summary>
/// Полный размер глобальной системы
/// </summary>
[Key(6)]
public int GlobalSize { get; set; }

public MatrixBlock() { }

public MatrixBlock(int startRow, int startCol, int rowCount, int colCount, int globalSize)
{
    StartRow = startRow;
    StartCol = startCol;
    RowCount = rowCount;
    ColCount = colCount;
    GlobalSize = globalSize;
    Data = new double[rowCount, colCount];
    LocalB = new double[rowCount];
}

/// <summary>
/// Получить элемент блока по локальным индексам
/// </summary>
public double GetElement(int localRow, int localCol) => Data[localRow, localCol];

/// <summary>
/// Установить элемент блока по локальным индексам
/// </summary>
public void SetElement(int localRow, int localCol, double value) => Data[localRow, localCol] = value;

/// <summary>
/// Получить глобальный индекс строки
/// </summary>
public int ToGlobalRow(int localRow) => StartRow + localRow;
```

```
    /// <summary>
    /// Получить глобальный индекс столбца
    /// </summary>
    public int ToGlobalCol(int localCol) => StartCol + localCol;
}
namespace DistributedSLAU.Common;

/// <summary>
/// Генератор тестовых матриц для СЛАУ
/// </summary>
public static class MatrixGenerator
{
    private static readonly Random DefaultRandom = new(42);

    /// <summary>
    /// Генерация случайной диагонально-доминантной матрицы (хорошо обусловленной)
    /// </summary>
    public static LinearSystem GenerateDiagonallyDominant(int size, double diagonalFactor = 2.0, int? seed = null)
    {
        var random = seed.HasValue ? new Random(seed.Value) : DefaultRandom;
        var system = new LinearSystem(size);

        for (int i = 0; i < size; i++)
        {
            double rowSum = 0;

            for (int j = 0; j < size; j++)
            {
                if (i != j)
                {
                    system.Matrix[i, j] = (random.NextDouble() - 0.5) * 10;
                    rowSum += Math.Abs(system.Matrix[i, j]);
                }
            }

            // Диагональное преобладание для устойчивости
```

```
            system.Matrix[i, i] = rowSum * diagonalFactor + random.NextDouble() * 5 + 1;
            system.VectorB[i] = (random.NextDouble() - 0.5) * 100;
        }

        return system;
    }

    /// <summary>
    /// Генерация матрицы с известным решением (для проверки точности)
    /// </summary>
    public static (LinearSystem system, double[] knownSolution) GenerateWithKnownSolution(int size, int? seed =
null)
    {
        var random = seed.HasValue ? new Random(seed.Value) : DefaultRandom;

        // Генерируем известное решение
        var knownSolution = new double[size];
        for (int i = 0; i < size; i++)
        {
            knownSolution[i] = (random.NextDouble() - 0.5) * 20;
        }

        // Генерируем диагонально-доминантную матрицу
        var system = GenerateDiagonallyDominant(size, 2.0, seed);

        // Вычисляем вектор b = A * x_known
        for (int i = 0; i < size; i++)
        {
            double sum = 0;
            for (int j = 0; j < size; j++)
            {
                sum += system.Matrix[i, j] * knownSolution[j];
            }
            system.VectorB[i] = sum;
        }

        return (system, knownSolution);
```

```
}

/// <summary>
/// Генерация плотной матрицы без специальных свойств
/// </summary>
public static LinearSystem GenerateDense(int size, int? seed = null)
{
    var random = seed.HasValue ? new Random(seed.Value) : DefaultRandom;
    var system = new LinearSystem(size);

    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            system.Matrix[i, j] = (random.NextDouble() - 0.5) * 20;
        }
        // Добавляем немного к диагонали для избежания вырожденности
        system.Matrix[i, i] += size * 2;
        system.VectorB[i] = (random.NextDouble() - 0.5) * 100;
    }

    return system;
}

/// <summary>
/// Генерация ленточной матрицы (для тестирования производительности)
/// </summary>
public static LinearSystem GenerateBanded(int size, int bandwidth, int? seed = null)
{
    var random = seed.HasValue ? new Random(seed.Value) : DefaultRandom;
    var system = new LinearSystem(size);

    for (int i = 0; i < size; i++)
    {
        int startCol = Math.Max(0, i - bandwidth);
        int endCol = Math.Min(size - 1, i + bandwidth);
```

```
        for (int j = 0; j < size; j++)
        {
            if (j >= startCol && j <= endCol)
            {
                system.Matrix[i, j] = (random.NextDouble() - 0.5) * 10;
            }
        }

        system.Matrix[i, i] += bandwidth * 2 + 5;
        system.VectorB[i] = (random.NextDouble() - 0.5) * 50;
    }

    return system;
}

/// <summary>
/// Сохранение матрицы в файл
/// </summary>
public static void SaveToFile(LinearSystem system, string matrixPath, string vectorPath, bool append = false)
{
    using var matrixWriter = new StreamWriter(matrixPath, append);
    using var vectorWriter = new StreamWriter(vectorPath, append);

    if (append)
    {
        matrixWriter.WriteLine();
        vectorWriter.WriteLine();
    }

    matrixWriter.WriteLine(system.Size);
    for (int i = 0; i < system.Size; i++)
    {
        var row = new List<string>();
        for (int j = 0; j < system.Size; j++)
        {
            row.Add(system.Matrix[i, j].ToString("G17", System.Globalization.CultureInfo.InvariantCulture));
        }
```

```
            matrixWriter.WriteLine(string.Join(" ", row));
        }

        var vectorValues = new List<string>();
        for (int i = 0; i < system.Size; i++)
        {
            vectorValues.Add(system.VectorB[i].ToString("G17", System.Globalization.CultureInfo.InvariantCulture));
        }
        vectorWriter.WriteLine(string.Join(" ", vectorValues));
    }

    /// <summary>
    /// Загрузка матрицы из файла
    /// </summary>
    public static LinearSystem LoadFromFile(string matrixPath, string vectorPath, int matrixIndex = 0)
    {
        string matrixContent = File.ReadAllText(matrixPath);
        string vectorContent = File.ReadAllText(vectorPath);

        string[] matrixBlocks = matrixContent.Split(new[] { "\n\n", "\r\n\r\n" },
StringSplitOptions.RemoveEmptyEntries);
        string[] vectorBlocks = vectorContent.Split(new[] { "\n\n", "\r\n\r\n" },
StringSplitOptions.RemoveEmptyEntries);

        if (matrixIndex >= matrixBlocks.Length)
            matrixIndex = 0;

        string[] matrixLines = matrixBlocks[matrixIndex].Split(new[] { '\n', '\r' },
StringSplitOptions.RemoveEmptyEntries);
        int size = int.Parse(matrixLines[0].Trim());

        var system = new LinearSystem(size);
        var culture = System.Globalization.CultureInfo.InvariantCulture;

        for (int i = 0; i < size; i++)
        {
            string[] parts = matrixLines[i + 1].Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);
            for (int j = 0; j < size; j++)
```

```
            {
                system.Matrix[i, j] = double.Parse(parts[j].Replace(',', '.'), culture);
            }
        }

        string[] vectorParts = vectorBlocks[matrixIndex].Split(new[] { ' ', '\t' },
StringSplitOptions.RemoveEmptyEntries);
        for (int i = 0; i < size; i++)
        {
            system.VectorB[i] = double.Parse(vectorParts[i].Replace(',', '.'), culture);
        }

        return system;
    }
}
namespace DistributedSLAU.Common;

/// <summary>
/// Разбиение матрицы на клетки (блоки) для распределённой обработки
/// </summary>
public static class MatrixPartitioner
{
    /// <summary>
    /// Разбивает матрицу на квадратные клетки (блоковое разбиение)
    /// </summary>
    /// <param name="system">Исходная СЛАУ</param>
    /// <param name="gridSize">Размер сетки (gridSize x gridSize блоков)</param>
    /// <returns>Двумерный массив блоков</returns>
    public static MatrixBlock[,] PartitionIntoBlocks(LinearSystem system, int gridSize)
    {
        int n = system.Size;
        int blockSize = (n + gridSize - 1) / gridSize; // округление вверх

        var blocks = new MatrixBlock[gridSize, gridSize];

        for (int blockRow = 0; blockRow < gridSize; blockRow++)
        {
```

```
        for (int blockCol = 0; blockCol < gridSize; blockCol++)
        {
            int startRow = blockRow * blockSize;
            int startCol = blockCol * blockSize;
            int rowCount = Math.Min(blockSize, n - startRow);
            int colCount = Math.Min(blockSize, n - startCol);

            if (rowCount <= 0 || colCount <= 0) continue;

            var block = new MatrixBlock(startRow, startCol, rowCount, colCount, n);

            // Копируем данные из глобальной матрицы в блок
            for (int i = 0; i < rowCount; i++)
            {
                for (int j = 0; j < colCount; j++)
                {
                    block.Data[i, j] = system.Matrix[startRow + i, startCol + j];
                }
                block.LocalB[i] = system.VectorB[startRow + i];
            }

            blocks[blockRow, blockCol] = block;
        }
    }

    return blocks;
}

/// <summary>
/// Разбивает матрицу на горизонтальные полосы (для сравнения)
/// </summary>
public static MatrixBlock[] PartitionByRows(LinearSystem system, int nodeCount)
{
    int n = system.Size;
    var blocks = new List<MatrixBlock>();

    int rowsPerNode = n / nodeCount;
```

```
    int remainder = n % nodeCount;

    int currentRow = 0;
    for (int i = 0; i < nodeCount; i++)
    {
        int rowsForNode = rowsPerNode + (i < remainder ? 1 : 0);
        if (rowsForNode <= 0) continue;

        var block = new MatrixBlock(currentRow, 0, rowsForNode, n, n);

        for (int r = 0; r < rowsForNode; r++)
        {
            int globalRow = currentRow + r;
            for (int col = 0; col < n; col++)
            {
                block.Data[r, col] = system.Matrix[globalRow, col];
            }
            block.LocalB[r] = system.VectorB[globalRow];
        }

        blocks.Add(block);
        currentRow += rowsForNode;
    }

    return blocks.ToArray();
}

/// <summary>
/// Собирает решение из блоков
/// </summary>
public static double[] GatherSolution(MatrixBlock[,] blocks, int gridSize)
{
    int n = blocks[0, 0].GlobalSize;
    var solution = new double[n];

    for (int i = 0; i < gridSize; i++)
    {
```

```
        for (int j = 0; j < gridSize; j++)
        {
            if (blocks[i, j] == null) continue;
            // Решение собирается из диагональных блоков
            if (i == j)
            {
                var block = blocks[i, j];
                for (int r = 0; r < block.RowCount; r++)
                {
                    // Будет перезаписано при получении решения
                }
            }
        }
    }

    return solution;
}

/// <summary>
/// Собирает решение из словаря частичных результатов
/// </summary>
public static double[] GatherSolutionFromParts(Dictionary<int, double[]> partialResults, int totalSize)
{
    var solution = new double[totalSize];

    foreach (var kvp in partialResults)
    {
        int startRow = kvp.Key;
        double[] localSolution = kvp.Value;

        for (int i = 0; i < localSolution.Length; i++)
        {
            solution[startRow + i] = localSolution[i];
        }
    }

    return solution;
```

```
    }
}

using System.Net;
using System.Net.Sockets;
using MessagePack;

namespace DistributedSLAU.Common;

/// <summary>
/// Базовый класс для UDP-коммуникации между узлами
/// </summary>
public abstract class UdpCommunicator : IDisposable
{
    protected UdpClient udpClient;
    protected CancellationTokenSource cts;
    protected Task? listenerTask;
    protected readonly int port;
    protected readonly IPEndPoint listenEndPoint;

    public event Func<NetworkMessage, IPEndPoint, Task>? MessageReceived;

    protected UdpCommunicator(int localPort)
    {
        port = localPort;
        listenEndPoint = new IPEndPoint(IPAddress.Any, localPort);

        udpClient = new UdpClient();
        udpClient.Client.SetSocketOption(SocketOptionLevel.Socket, SocketOptionName.ReuseAddress, true);
        udpClient.Client.Bind(listenEndPoint);

        cts = new CancellationTokenSource();

        Console.WriteLine($"[UDP] Слушаем на порту {localPort}");
    }
```

```
public void StartListening()
{
    listenerTask = Task.Run(ListenLoop);
}

private async Task ListenLoop()
{
    while (!cts.Token.IsCancellationRequested)
    {
        try
        {
            var result = await udpClient.ReceiveAsync();
            _ = Task.Run(() => ProcessMessage(result.Buffer, result.RemoteEndPoint));
        }
        catch (ObjectDisposedException)
        {
            break;
        }
        catch (Exception ex)
        {
            Console.WriteLine($"[UDP] Ошибка приема: {ex.Message}");
        }
    }
}

private async Task ProcessMessage(byte[] data, IPEndPoint remoteEndPoint)
{
    try
    {
        var message = MessagePackSerializer.Deserialize<NetworkMessage>(data);

        if (MessageReceived != null)
        {
            await MessageReceived.Invoke(message, remoteEndPoint);
        }
    }
    catch (Exception ex)
```

```
    {
        Console.WriteLine($"[UDP] Ошибка десериализации: {ex.Message}");
    }
}

public async Task SendToAsync(NetworkMessage message, string ipAddress, int port)
{
    try
    {
        var data = MessagePackSerializer.Serialize(message);
        var endpoint = new IPEndPoint(IPAddress.Parse(ipAddress), port);
        await udpClient.SendAsync(data, data.Length, endpoint);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[UDP] Ошибка отправки: {ex.Message}");
    }
}

public async Task BroadcastAsync(NetworkMessage message, int port)
{
    try
    {
        var data = MessagePackSerializer.Serialize(message);
        udpClient.EnableBroadcast = true;
        var endpoint = new IPEndPoint(IPAddress.Broadcast, port);
        await udpClient.SendAsync(data, data.Length, endpoint);
        udpClient.EnableBroadcast = false;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[UDP] Ошибка широковещательной отправки: {ex.Message}");
    }
}

public virtual void Stop()
{
```

```
        cts.Cancel();
        udpClient?.Close();
        udpClient?.Dispose();
        listenerTask?.Wait(1000);
    }

    public void Dispose()
    {
        Stop();
        udpClient?.Dispose();
    }
}
namespace DistributedSLAU.Common;

/// <summary>
/// Информация о вычислительном узле
/// </summary>
public class WorkerNode
{
    public string IpAddress { get; set; }
    public int Port { get; set; }
    public string Id => $"{IpAddress}:{Port}";
    public bool IsAlive { get; set; } = true;
    public DateTime LastHeartbeat { get; set; } = DateTime.UtcNow;

    public WorkerNode(string ipAddress, int port)
    {
        IpAddress = ipAddress;
        Port = port;
    }

    public override string ToString() => Id;
}
using System.Diagnostics;
using System.Net;
using MessagePack;
using DistributedSLAU.Common;
```

```
namespace DistributedSLAU.Coordinator;

/// <summary>
/// Координатор для распределённого решения СЛАУ
/// </summary>
public class DistributedCoordinator : UdpCommunicator
{
    private readonly List<WorkerNode> workers = new();
    private LinearSystem? currentSystem;

    // Результаты
    private readonly Dictionary<int, double[]> partialSolutions = new();
    private readonly Dictionary<(int row, int col), MatrixBlock> processedBlocks = new();
    private readonly object lockObj = new();

    // Синхронизация
    private TaskCompletionSource<bool>? completionTcs;
    private int expectedResponses;
    private int receivedResponses;

    public DateTime StartTime { get; private set; }

    public DistributedCoordinator(int port) : base(port)
    {
        MessageReceived += OnMessageReceived;
    }

    private async Task OnMessageReceived(NetworkMessage message, IPEndPoint remoteEndPoint)
    {
        switch (message.Type)
        {
            case MessageType.Ready:
                await HandleReady(message, remoteEndPoint);
                break;

            case MessageType.PartialResult:
```

```
            await HandlePartialResult(message, remoteEndPoint);
            break;

        case MessageType.BlockResult:
            await HandleBlockResult(message, remoteEndPoint);
            break;

        case MessageType.Heartbeat:
            await SendToAsync(new NetworkMessage(MessageType.Heartbeat, null),
                remoteEndPoint.Address.ToString(), remoteEndPoint.Port);
            break;
    }
}

private Task HandleReady(NetworkMessage message, IPEndPoint remoteEndPoint)
{
    var workerId = $"{remoteEndPoint.Address}:{remoteEndPoint.Port}";
    Console.WriteLine($"[Coordinator] Получено Ready от {workerId}");
    return Task.CompletedTask;
}

private Task HandlePartialResult(NetworkMessage message, IPEndPoint remoteEndPoint)
{
    if (message.Data == null) return Task.CompletedTask;

    var solution = MessagePackSerializer.Deserialize<double[]>(message.Data);

    lock (lockObj)
    {
        partialSolutions[message.BlockRowIndex] = solution;
        receivedResponses++;

        Console.WriteLine($"[Coordinator] Получено от {remoteEndPoint.Port}: строки
{message.BlockRowIndex} ({receivedResponses}/{expectedResponses})");

        if (receivedResponses >= expectedResponses)
        {
```

```
            completionTcs?.TrySetResult(true);
        }
    }

    return Task.CompletedTask;
}

private Task HandleBlockResult(NetworkMessage message, IPEndPoint remoteEndPoint)
{
    if (message.Data == null) return Task.CompletedTask;

    lock (lockObj)
    {
        var block = MessagePackSerializer.Deserialize<MatrixBlock>(message.Data);
        processedBlocks[(message.BlockRowIndex, message.BlockColIndex)] = block;
        receivedResponses++;

        Console.WriteLine($"[Coordinator] Получен блок ({message.BlockRowIndex},{message.BlockColIndex})
от {remoteEndPoint.Port} ({receivedResponses}/{expectedResponses})");

        if (receivedResponses >= expectedResponses)
        {
            completionTcs?.TrySetResult(true);
        }
    }

    return Task.CompletedTask;
}

/// <summary>
/// Загрузка списка рабочих узлов из файла
/// </summary>
public void LoadWorkers(string filePath)
{
    workers.Clear();
    var lines = File.ReadAllLines(filePath);
```

```
    foreach (var line in lines)
    {
        if (string.IsNullOrWhiteSpace(line)) continue;

        var parts = line.Split(':');
        if (parts.Length >= 2)
        {
            string ip = parts[0].Trim();
            if (int.TryParse(parts[1].Trim(), out int port))
            {
                var worker = new WorkerNode(ip, port);
                workers.Add(worker);
                Console.WriteLine($"[Coordinator] Добавлен узел: {worker.Id}");
            }
        }
    }

    Console.WriteLine($"[Coordinator] Всего загружено узлов: {workers.Count}");
}

/// <summary>
/// Распределённое решение
/// </summary>
public async Task<double[]> SolveDistributedAsync()
{
    if (workers.Count == 0)
        throw new InvalidOperationException("Нет доступных узлов!");

    if (currentSystem == null)
        throw new InvalidOperationException("Система не загружена!");

    return await SolveDistributedAsync(currentSystem);
}

/// <summary>
/// Распределённое решение
/// </summary>
```

```
public async Task<double[]> SolveDistributedAsync(LinearSystem system)
{
    if (workers.Count == 0)
        throw new InvalidOperationException("Нет доступных узлов!");

    int n = system.Size;
    int numWorkers = workers.Count;

    Console.WriteLine($"[Coordinator] Распределённое решение: {n}x{n}");
    Console.WriteLine($"[Coordinator] Воркеров: {numWorkers}");

    StartTime = DateTime.Now;

    // Если воркеров >= 4, используем клеточное разбиение
    if (numWorkers >= 4)
    {
        int gridSize = Math.Min((int)Math.Ceiling(Math.Sqrt(numWorkers)),
                                Math.Min(n, 4));
        int blockSize = (n + gridSize - 1) / gridSize;
        return await SolveByCellularGauss(system, gridSize, blockSize);
    }
    else
    {
        // Иначе используем разбиение по строкам
        return await SolveByRows(system, numWorkers);
    }
}

/// <summary>
/// Решение с разбиением по строкам
/// </summary>
private async Task<double[]> SolveByRows(LinearSystem system, int numWorkers)
{
    int n = system.Size;
    int rowsPerWorker = (n + numWorkers - 1) / numWorkers;

    Console.WriteLine($"[Coordinator] Разбиение по строкам: ~{rowsPerWorker} строк на воркер");
```

```
partialSolutions.Clear();
receivedResponses = 0;
expectedResponses = numWorkers;
completionTcs = new TaskCompletionSource<bool>();

// Отправляем полную систему каждому воркеру — ПАРАЛЛЕЛЬНО!
var sendTasks = new List<Task>();
for (int i = 0; i < numWorkers; i++)
{
    var worker = workers[i];
    int workerStartRow = i * rowsPerWorker;
    int workerRowCount = Math.Min(rowsPerWorker, n - workerStartRow);

    if (workerRowCount <= 0) continue;

    var data = MessagePackSerializer.Serialize(system);
    var message = new NetworkMessage(MessageType.TaskDistribution, data)
    {
        BlockRowIndex = workerStartRow,
        BlockColIndex = workerRowCount
    };

    Console.WriteLine($"[Coordinator] >>> Отправляю систему узлу {worker.IpAddress}:{worker.Port} " +
                $"(строки {workerStartRow}-{workerStartRow + workerRowCount - 1})");
    sendTasks.Add(SendToAsync(message, worker.IpAddress, worker.Port));
}

await Task.WhenAll(sendTasks);
Console.WriteLine($"[Coordinator] Все системы отправлены, ждём результаты...");

// Ждём результаты (таймаут 10 секунд)
var timeoutTask = Task.Delay(10000);
var completedTask = await Task.WhenAny(completionTcs.Task, timeoutTask);

if (completedTask == timeoutTask)
{
```

```
            Console.WriteLine($"[Coordinator] ⚠ ТАЙМАУТ! Получено {receivedResponses}/{expectedResponses}");
            Console.WriteLine($"[Coordinator] Решаем последовательно...");
            return CellGaussSolver.SolveSequential(system);
        }

        Console.WriteLine($"[Coordinator] ✓ Все результаты получены
({receivedResponses}/{expectedResponses})");

        // Собираем решение
        var solution = new double[n];
        foreach (var kvp in partialSolutions.OrderBy(x => x.Key))
        {
            int startRow = kvp.Key;
            for (int i = 0; i < kvp.Value.Length && startRow + i < n; i++)
            {
                solution[startRow + i] = kvp.Value[i];
            }
        }

        var elapsed = DateTime.Now - StartTime;
        Console.WriteLine($"[Coordinator] ✓ Решение найдено за {elapsed.TotalMilliseconds} мс");

        return solution;
    }

    /// <summary>
    /// Клеточный метод Гаусса (2D разбиение)
    /// </summary>
    private async Task<double[]> SolveByCellularGauss(LinearSystem system, int gridSize, int blockSize)
    {
        int n = system.Size;
        Console.WriteLine($"[Coordinator] Клеточный метод Гаусса: сетка {gridSize}x{gridSize}");
        Console.WriteLine($"[Coordinator] Размер клетки: ~{blockSize}x{blockSize}");

        var blocks = MatrixPartitioner.PartitionIntoBlocks(system, gridSize);

        // Этап 1: Прямой ход
```

```
    Console.WriteLine("\n[Coordinator] === ЭТАП 1: Прямой ход ===");
    await PerformCellularForwardElimination(blocks, gridSize);

    // Этап 2: Обмен
    Console.WriteLine("\n[Coordinator] === ЭТАП 2: Обмен ===");
    await PerformBlockExchange(blocks, gridSize);

    // Этап 3: Обратный ход
    Console.WriteLine("\n[Coordinator] === ЭТАП 3: Обратный ход ===");
    var solution = await PerformCellularBackSubstitution(blocks, gridSize, n);

    var elapsed = DateTime.Now - StartTime;
    Console.WriteLine($"\n[Coordinator] ✅ Решение найдено за {elapsed.TotalMilliseconds} мс");

    return solution;
}

private async Task PerformCellularForwardElimination(MatrixBlock[,] blocks, int gridSize)
{
    processedBlocks.Clear();
    receivedResponses = 0;
    expectedResponses = gridSize * gridSize;
    completionTcs = new TaskCompletionSource<bool>();

    var sendTasks = new List<Task>();
    int workerIndex = 0;

    for (int i = 0; i < gridSize; i++)
    {
        for (int j = 0; j < gridSize; j++)
        {
            if (blocks[i, j] == null) continue;

            var worker = workers[workerIndex % workers.Count];
            var data = MessagePackSerializer.Serialize(blocks[i, j]);
            var message = new NetworkMessage(MessageType.BlockDistribution, data)
            {
```

```
                BlockRowIndex = i,
                BlockColIndex = j,
                GridSize = gridSize,
                Phase = 1
            };

            Console.WriteLine($"[Coordinator] >>> Отправляю клетку ({i},{j}) узлу
{worker.IpAddress}:{worker.Port}");
            sendTasks.Add(SendToAsync(message, worker.IpAddress, worker.Port));
            workerIndex++;
        }
    }

    await Task.WhenAll(sendTasks);
    Console.WriteLine($"[Coordinator] Все клетки отправлены, ждём обработки...");

    var timeoutTask = Task.Delay(10000);
    var completedTask = await Task.WhenAny(completionTcs.Task, timeoutTask);

    if (completedTask == timeoutTask)
    {
        Console.WriteLine($"[Coordinator] ⚠ ТАЙМАУТ! Получено {receivedResponses}/{expectedResponses}");
    }
    else
    {
        Console.WriteLine($"[Coordinator] ✅ Этап 1 завершён: {receivedResponses}/{expectedResponses}");
    }

    // Локальная обработка для гарантии
    for (int i = 0; i < gridSize; i++)
        for (int j = 0; j < gridSize; j++)
            if (blocks[i, j] != null)
                CellGaussSolver.BlockForwardElimination(blocks[i, j]);
}

private Task PerformBlockExchange(MatrixBlock[,] blocks, int gridSize)
{
```

```
    Console.WriteLine("[Coordinator] Обмен завершён");
    return Task.CompletedTask;
}

private async Task<double[]> PerformCellularBackSubstitution(MatrixBlock[,] blocks, int gridSize, int n)
{
    var fullMatrix = new double[n, n];
    var fullVector = new double[n];

    for (int i = 0; i < gridSize; i++)
    {
        for (int j = 0; j < gridSize; j++)
        {
            if (blocks[i, j] != null)
            {
                var block = blocks[i, j];
                for (int bi = 0; bi < block.RowCount; bi++)
                {
                    for (int bj = 0; bj < block.ColCount; bj++)
                    {
                        fullMatrix[block.StartRow + bi, block.StartCol + bj] = block.Data[bi, bj];
                    }
                    fullVector[block.StartRow + bi] = block.LocalB[bi];
                }
            }
        }
    }

    var solution = new double[n];
    for (int i = n - 1; i >= 0; i--)
    {
        double sum = 0;
        for (int j = i + 1; j < n; j++)
            sum += fullMatrix[i, j] * solution[j];

        solution[i] = Math.Abs(fullMatrix[i, i]) < 1e-12 ? 0 : (fullVector[i] - sum) / fullMatrix[i, i];
    }
```

```
        Console.WriteLine("[Coordinator] ✅ Обратный ход завершён");
        return solution;
    }

    /// <summary>
    /// Загрузка системы из файлов
    /// </summary>
    public LinearSystem LoadSystem(string matrixFile, string vectorFile, int matrixIndex = 0)
    {
        string matrixContent = File.ReadAllText(matrixFile);
        string vectorContent = File.ReadAllText(vectorFile);

        string[] matrixBlocks = matrixContent.Split(new[] { "\n\n", "\r\n\r\n" },
StringSplitOptions.RemoveEmptyEntries);
        string[] vectorBlocks = vectorContent.Split(new[] { "\n\n", "\r\n\r\n" },
StringSplitOptions.RemoveEmptyEntries);

        if (matrixIndex >= matrixBlocks.Length)
            matrixIndex = 0;

        string[] matrixLines = matrixBlocks[matrixIndex].Split(new[] { '\n', '\r' },
StringSplitOptions.RemoveEmptyEntries);
        int size = int.Parse(matrixLines[0].Trim());

        var system = new LinearSystem(size);
        var culture = System.Globalization.CultureInfo.InvariantCulture;

        for (int i = 0; i < size; i++)
        {
            string line = matrixLines[i + 1].Trim();
            string[] parts = line.Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);

            for (int j = 0; j < size; j++)
            {
                system.Matrix[i, j] = double.Parse(parts[j].Replace(',', '.'), culture);
            }
        }
```

```
    string vectorLine = vectorBlocks[matrixIndex].Trim();
    string[] vectorParts = vectorLine.Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);

    for (int i = 0; i < size; i++)
    {
        system.VectorB[i] = double.Parse(vectorParts[i].Replace(',', '.'), culture);
    }

    currentSystem = system;
    Console.WriteLine($"Система #{matrixIndex + 1} загружена! Размер: {size}x{size}");
    return system;
}

/// <summary>
/// Последовательное решение
/// </summary>
public (double[] solution, TimeSpan elapsed) SolveSequential()
{
    if (currentSystem == null)
        throw new InvalidOperationException("Система не загружена");

    var sw = Stopwatch.StartNew();
    var solution = CellGaussSolver.SolveSequential(currentSystem);
    sw.Stop();

    return (solution, sw.Elapsed);
}

public void PrintWorkers()
{
    Console.WriteLine("\n=== Доступные узлы ===");
    foreach (var worker in workers)
        Console.WriteLine($"  {worker.Id}");
}

public void AddWorker(string ip, int port)
```

```
    {
        var worker = new WorkerNode(ip, port);
        if (!workers.Any(w => w.IpAddress == ip && w.Port == port))
        {
            workers.Add(worker);
            Console.WriteLine($"[Coordinator] Добавлен узел: {worker.Id}");
        }
    }
}
```